

A REPORT
ON
DESIGN AND IMPLEMENTATION OF AN OTA FIRMWARE
UPDATE SYSTEM FOR EMBEDDED DEVICES

BY
VARAD VINOD PRABHU
2024HT01040

AT
AUMOVIO India Private Limited
(Formerly known as Continental Automotive Components India Private Limited)
Bengaluru

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
May 2026

A REPORT
ON
DESIGN AND IMPLEMENTATION OF AN OTA FIRMWARE
UPDATE SYSTEM FOR EMBEDDED DEVICES

BY
VARAD VINOD PRABHU
2024HT01040
MTech. Embedded Systems

Prepared in partial fulfilment of the
WILP Dissertation Course

AT
AUMOVIO India Private Limited
(Formerly known as Continental Automotive Components India Private Limited)
Bengaluru

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
May 2026

ACKNOWLEDGEMENTS

I would like to express my gratitude to my supervisor, Subramanya N, for their continuous guidance, technical support, and insight throughout the course of this dissertation work. Their valuable comments greatly contributed to the successful completion of this project.

I would also like to thank the additional examiner, Amit Nadkarni, for their valuable feedback and support during the evaluation of this work.

I also extend my thanks to my faculty mentor, Dr. Meetha V. Shenoy, for their vital feedback during my progress reviews, academic guidance and valuable suggestions throughout the dissertation.

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE, PILANI

(RAJASTHAN)

WILP Division

Organization: AUMOVIO India Private Limited

Location: Bengaluru

Duration: 4 months

Date of Start: 1st January 2026

Date of Submission: 12th May 2026

Title of the Project: Design and implementation of an OTA firmware update system for embedded devices

Name of the Student: Varad Vinod Prabhu

ID No.: 2024HT01040

Name and Designation of Supervisor: Subramanya N, Technical Architect

Name and Designation of Additional Examiner: Amit Nadkarni, Senior Manager

Name of the Faculty mentor: Dr. Meetha V. Shenoy

Key Words: Over-the-Air (OTA) Firmware Update, Secure Bootloader, Firmware Authentication, Rollback Protection, Cybersecurity.

Project Areas: Embedded Systems, Cybersecurity

Abstract:

Modern automotive Electronic Control Units (ECUs) are increasingly software-dependent, yet conventional firmware update practices require physical access to the vehicle at a service centre. This dissertation presents the design and implementation of a reliable Over-the-Air (OTA) firmware update

system for embedded controllers, targeting the STM32H753ZI microcontroller with the ESP32 acting as a low-cost Wi-Fi bridge.

The system exploits the dual-bank flash architecture of the STM32H753ZI, where the inactive bank receives the new firmware while the active bank continues executing, eliminating device downtime during the update process. A custom binary UART protocol operating at 115200 bps transfers firmware in 256-byte packets with CRC-16-CCITT integrity. Each firmware image is accompanied by a structured header carrying its SHA-256 hash, ECDSA signature and version number, which the STM32 application verifies upon transfer completion.

A bare-metal custom bootloader, mirrored across both flash banks, executes on every power-on or reset event and implements a boot decision algorithm based on per-bank firmware metadata. The bootloader selects the higher-versioned valid firmware and performs a hardware bank swap by programming the STM32H753ZI SWAP_BANK option byte when required. A watchdog-driven rollback mechanism detects firmware failures at runtime.

Results confirm that the system reliably delivers, validates, and activates firmware updates while recovering from failure conditions, addressing a gap identified across prior work in combining dual-bank hardware, watchdog rollback, and wireless delivery within a single low-cost prototype.



Signature of the Student

Date: 11th May 2026



Signature of the Supervisor

Date: 12th May 2026

Thank You Team
EVM
BITS Pilani WILP

Contents

1. Introduction.....	1
1.1. Broad Area of Work.....	1
1.2. Background.....	2
1.3. Objectives	3
1.4. Scope of Work	4
2. Literature Survey	5
3. Implementation Details.....	10
3.1. Methodology	10
3.1.1. Flash memory map of STM32H753ZI	10
3.1.2. Custom Bootloader	12
3.1.3. Packet Format	14
3.1.4. Security Algorithm Selection	17
3.2. Design Overview	18
3.3. Tools and Platform.....	20
3.4. Assumption or Constraints.....	21

4. Results and Discussion	22
4.1. Hardware setup	22
4.2. Secure bootloader performance	23
4.3. OTA update pipeline performance.....	25
4.4. Functional test results	27
4.5. Summary	43
5. Conclusion and Future Scope of Work.....	44
6. Abbreviations	46
7. References.....	47

List of Figures

1. Figure 1 Custom bootloader flowchart	12
2. Figure 2 Block diagram of implementation	19
3. Figure 3 Hardware setup	23
4. Figure 4 Test case 1: No valid firmware in each bank	28
5. Figure 5 Test case 2: Valid firmware in Bank 1 and Bank 2 is empty	29
6. Figure 6 Test case 3: Normal OTA update: Bank 1 active, new firmware received	29
7. Figure 7 Test case 4a: OTA update with SHA-256 hash corrupted	30
8. Figure 8 Test case 4b: OTA update with SHA-256 hash corrupted	31
9. Figure 9 Test case 5a: OTA update with ECDSA signature corrupted	32
10. Figure 10 Test case 5b: OTA update with ECDSA signature corrupted	32
11. Figure 11 Test case 6a: OTA update with wrong nonce for AES decryption	33
12. Figure 12 Test case 6b: OTA update with wrong nonce for AES decryption	33
13. Figure 13 Test case 7a: Valid OTA update but updated firmware triggers watchdog reset	34
14. Figure 14 Test case 7b: Valid OTA update but updated firmware triggers watchdog reset	35
15. Figure 15 Test case 8: Firmware transfer interrupted	36
16. Figure 16 Test case 9a(i): Secure boot integrity verification failed due to invalid SHA-256 hash	37
17. Figure 17 Test case 9a(ii): Secure boot integrity verification failed due to invalid SHA-256 hash ...	37
18. Figure 18 Test case 9b(i): Secure boot integrity verification failed due to invalid SHA-256 hash	38
19. Figure 19 Test case 9b(ii): Secure boot integrity verification failed due to invalid SHA-256 hash ...	39
20. Figure 20 Test case 10a(i): Secure boot authentication failed due to invalid ECDSA signature	40
21. Figure 21 Test case 10a(ii): Secure boot authentication failed due to invalid ECDSA signature	41
22. Figure 22 Test case 10b(i): Secure boot authentication failed due to invalid ECDSA signature	42

23. Figure 23 Test case 10b(ii): Secure boot authentication failed due to invalid ECDSA signature.....	42
--	----

List of Tables

1. Table 1 Flash memory map of STM32H753ZI	10
2. Table 2 Packet Structure	15
3. Table 3 Secure bootloader cryptographic performance (firmware size: 128 KB).....	23
4. Table 4 Secure bootloader cryptographic performance for various firmware sizes	25
5. Table 5 OTA update timing comparison software vs hardware accelerated (firmware size: 128 KB)	25
6. Table 6 Abbreviations.....	46

1. Introduction

1.1. Broad Area of Work

This dissertation presents the design and implementation of a secure, reliable Over-the-Air (OTA) firmware update mechanism for automotive Electronic Control Units (ECUs). Modern vehicles are increasingly governed by software distributed across dozens of ECUs that collectively manage functions ranging from engine control and anti-lock braking to advanced driver-assistance systems (ADAS). A contemporary premium vehicle may contain over 100 ECUs executing upwards of 100 million lines of code, making continuous software maintenance a critical operational requirement throughout the vehicle lifecycle.

The work addresses this challenge through a complete embedded-level OTA system built around the STM32H753ZI microcontroller. The system encompasses a custom bare-metal bootloader with dual-bank flash architecture for zero-downtime updates, a hardware-accelerated cryptographic security chain for firmware integrity and confidentiality, and a deterministic watchdog-based rollback mechanism for safe recovery from failed updates. Firmware delivery is handled via an ESP32 Wi-Fi bridge module, simulating a wireless remote update channel typical of connected vehicle deployments.

A central contribution of this work is the integration of the STM32H7 hardware cryptographic peripherals, the CRYP peripheral for AES-256-CTR decryption and the HASH peripheral for SHA-256 integrity verification, alongside software-based ECDSA P-256 signature authentication using the mbedTLS library. This combination achieves a performance-efficient, layered security pipeline and provides a quantitative basis for comparing hardware-accelerated versus software-only cryptographic approaches in resource-constrained automotive ECUs. The implementation is

further validated through a FreeRTOS-based application simulating a brake ECU workload running concurrently with the OTA task, demonstrating real-world viability.

1.2. Background

The automotive industry has undergone a profound shift over the past two decades, transitioning from predominantly mechanical systems to software-defined platforms. ECUs now govern safety-critical subsystems such as electronic stability control, powertrain management, and semi-autonomous driving features. The complexity of this software creates a continuous demand for post-deployment updates to address defects, patch cybersecurity vulnerabilities, and introduce new functionality, often without the possibility of taking the vehicle off-road.

Traditionally, firmware updates for automotive ECUs are performed manually by technicians during service center visits. This manual firmware update approach results in higher maintenance costs, requires the vehicle to be physically present at the service center, and offers limited flexibility to update the software improvements. To have improved vehicle maintenance efficiency and reduce the operational costs, there is a growing demand for remote firmware update mechanisms.

Designing a remote firmware system for automotive application present several challenges due to the ECUs operating conditions. Any failure in the firmware update process, such as power interruption or corrupted firmware transfer can lead to ECU malfunction and impact the vehicle functionality. Therefore, there is a need to design a safe and reliable firmware update system which can recover from update failures. Physical recall campaigns for software-only defects have historically cost manufacturers hundreds of millions of dollars and affected millions of vehicles, highlighting the economic case for remote update capability.

Over-the-Air (OTA) firmware update mechanisms are used to update the ECU firmware remotely.

Remote firmware delivery introduces a significant attack surface. An adversary who can inject a malicious firmware image into an ECU could compromise vehicle safety systems with severe consequences. OTA systems must satisfy three fundamental security properties: confidentiality (preventing interception of firmware images), integrity (detecting corruption or tampering), and authenticity (verifying that firmware originates from a trusted source). Elliptic Curve Digital Signature Algorithm (ECDSA) with P-256 has become the preferred authentication primitive for automotive applications due to its compact key size and computational efficiency relative to RSA. AES-256 in Counter (CTR) mode is widely adopted for symmetric encryption of firmware payloads. SHA is used to detect the integrity of the firmware.

Reliability is equally critical along with security. Power interruptions during flash programming or corrupted data transfers can leave an ECU in an unbootable state, potentially immobilizing the vehicle. This project focuses on design and implementation of reliable OTA firmware update mechanism for automotive embedded systems. The work emphasizes on firmware reception, validation, and safe activation of firmware updates for automotive ECUs.

1.3.Objectives

- Analyse the limitations of conventional in-place ECU firmware update approaches for a remote OTA firmware update system for automotive ECUs.
- Design and implement a dual-bank flash memory architecture on the STM32H753ZI that enables background firmware reception into the inactive bank and atomic firmware activation without interrupting active ECU operation.
- Implement a hardware-accelerated OTA firmware security pipeline.
- Develop a secure bare-metal bootloader that verifies the firmware.

- Measure and compare the execution time of hardware-accelerated and software-only implementations to quantify the performance benefit of hardware cryptographic acceleration.
- Validate the implemented system through functional and failure test scenarios.

1.4.Scope of Work

The scope of this project includes the design and implementation of a secure OTA firmware update system for automotive ECUs using STM32H753ZI microcontroller. The project focuses on developing a custom secure bootloader with dual-bank flash architecture to support safe firmware updates and rollback mechanisms.

The system enables firmware transfer via ESP32 communication module and ensures firmware integrity through SHA validation. Additionally, security features such as AES-based encryption and ECDSA-based digital signature verification are implemented to ensure confidentiality and authenticity of firmware updates.

The project also includes testing under various scenarios such as power failure, invalid firmware, and rollback conditions.

The scope is limited to embedded-level implementation and does not include cloud-based OTA infrastructure, automotive safety certification, or real vehicle deployment. Partial firmware updates or delta patching is not a part of this project.

2. Literature Survey

The increasing complexity of embedded systems deployed in automotive and industrial environments has created a critical need for reliable, secure, and efficient mechanisms to update firmware in the field without physical access to the device. This section reviews relevant published works addressing firmware-over-the-air update systems for embedded controllers, with particular focus on protocol design, security mechanisms, dual-bank flash management, and automotive domain applicability. Each work is evaluated for its contributions, limitations, and the specific gaps that the present implementation addresses.

Mirfakhraie et al. [1] propose the MCC-FOTA protocol for updating automotive HVAC ECU firmware using cellular connectivity and a CAN bus network conforming to SAE J1939. The system uses a Fleet Tracker module that downloads firmware from an FTP server via a cellular gateway and transfers it to target ECUs over CAN bus using the XMODEM-1K file transfer protocol, which provides 1 KB packet transfers with CRC checking and ACK/NACK responses. The protocol divides firmware into 256-byte pages and includes a signature verification step to detect session hijack attacks. The system was validated on real HVAC ECUs with transfer times measured across multiple baud rates. This work provides direct motivation for the automotive ECU update use case in the present implementation and demonstrates the viability of using a dedicated gateway module to bridge the network and the ECU. No dual-bank flash mechanism is described, meaning the ECU is non-operational during reprogramming, and there is no automatic runtime failure detection or rollback. The present work addresses these gaps by implementing dual-bank flash hardware support and a watchdog-driven rollback state machine on the STM32H753ZI, using a low-cost ESP32 Wi-Fi bridge in place of a cellular gateway.

Kachman and Balaz [2] present a lightweight patch module for differential firmware updates on embedded devices, providing a detailed analysis and comparison of single-bank and dual-bank update strategies. In the single-bank approach, firmware is rewritten directly in its destination location, making it unavailable during programming. In the dual-bank approach, firmware is reconstructed in a second memory region and copied to its destination only after successful reconstruction, ensuring the previous firmware remains available if the update fails mid-way. The patch module is implemented in C and evaluated on a Nordic nRF52840 SoC, demonstrating that single-bank updates require 28 to 63 percent less flash memory than dual-bank, though at the cost of recovery capability. CRC-16 is used to verify delta file integrity. This work directly motivates the dual-bank architecture adopted in the present implementation, where the inactive flash region receives the new firmware while the active region continues executing. However, Kachman and Balaz focus exclusively on differential updates and the update processing stage, explicitly stating that networking and security are outside their scope. No wireless delivery, runtime failure detection, or automatic rollback is addressed. The present work extends the dual-bank concept with a complete boot algorithm, hardware watchdog rollback, and Wi-Fi firmware delivery.

S. K et al. [3] propose a FOTA system using Raspberry Pi 4 as a gateway, ESP8266 as the primary IoT node, and STM32F103C8T6 as the secondary node that receives firmware. The architecture uses IPFS for decentralized firmware storage, hybrid ECC and AES encryption for firmware confidentiality, gzip compression, SHA-256 for integrity verification, and MQTT with TLS/SSL for gateway-to-device communication. The ESP8266 receives the encrypted firmware, verifies the hash, and forwards it block by block to the STM32 with per-block CRC verification. The STM32 flash memory is partitioned into two 15 KB slots for storing previous and current firmware versions alongside a 16 KB bootloader region. This work directly inspires the bridge-based architecture in

the present implementation, where a wireless device handles network communication and forwards firmware to the STM32 over a secondary protocol. The hybrid encryption and SHA-256 integrity approach also inform the security design. However, the STM32F103C8T6 does not support hardware dual-bank flash with automatic bank swapping, and no hardware watchdog rollback is implemented. The Raspberry Pi gateway is not suitable for embedded deployment due to power and cost requirements. The present work replaces the Raspberry Pi with a low-cost ESP32, targets the STM32H753ZI with native hardware, and implements automatic rollback through watchdog hardware reset detection.

Kubaščík et al. [4] investigate OTA firmware update mechanisms for ESP32 microcontrollers using three wireless protocols Wi-Fi via HTTPS, Bluetooth Low Energy, and Zigbee all implemented through the Espressif IDF library. The paper explains the ESP32 partition table structure, where an OTA data partition points to the active application partition, and new firmware is written to the inactive partition during an update. The Espressif library supports rollback by reverting the OTA data pointer to the previous partition if the new firmware is found defective. The paper also proposes a future automated deployment network using a central server managing updates across all three protocols and discusses fault detection algorithms for version stability evaluation. This work provides understanding of the ESP32 OTA capabilities and partition management that informs the design decision to use the ESP32 as a bridge device in the present implementation. Crucially, Kubaščík et al. update the ESP32 itself as the OTA target, whereas the present work uses the ESP32 as an intermediate relay that downloads firmware from a server and retransmits it to the STM32H753ZI over a custom UART protocol.

Sangrulkar et al. [5] implement a secure and scalable OTA system for the STM32F407VG using a Quectel M66 GSM module for cellular connectivity, AWS IoT with MQTT for cloud

communication, and Azure Blob Storage as a firmware repository. The STM32 and M66 communicate via UART. Firmware is downloaded by the M66 from Azure Blob over HTTPS and transferred to the STM32. A bootloader on the STM32F407VG manages an update flag to detect pending updates and verifies firmware integrity using CRC before committing to flash. End-to-end encryption and digital signature verification are described as security measures. This is the most architecturally similar work to the present implementation both use an STM32 target, a wireless module communicating over UART, and a remote server. The bootloader update flag concept informs the metadata-based boot decision in the present work. However, the STM32F407VG belongs to the F4 series which lacks hardware dual-bank flash, making seamless bank switching unavailable. The Quectel M66 GSM module requires a SIM card and cellular subscription, increasing cost and complexity compared to Wi-Fi. No hardware watchdog rollback is implemented. The present work targets the STM32H753ZI with native dual-bank hardware, replaces cellular with a low-cost ESP32 Wi-Fi bridge, and implements automatic rollback through hardware watchdog reset cause analysis.

Wang et al. [6] design and validate an automotive OTA system using PKI-based hybrid encryption combining SHA-3 for hashing, RSA for digital signing and AES key encryption, and AES for firmware confidentiality. The system architecture includes an OTA server, Signature Verification Server, Key Management System, Certificate Authority, T-BOX telematics gateway, and ECU terminals connected via Automotive Ethernet. The update process is validated on real test vehicles by comparing update time against a traditional gateway-storage architecture. The proposed direct-download design reduces the update time for a 2.1 GB In-Vehicle Infotainment (IVI) package from 71.2 minutes to 20.9 minutes by allowing ECUs to download directly from the OTA server through the T-BOX without storing the package in the gateway. Tamper tests confirm that both signature

modification and package content replacement cause verification failure and prevent update execution. This work directly motivates the security architecture in the present implementation, particularly the combination of hash-based integrity verification and asymmetric signing. No bootloader design, dual-bank management, or rollback mechanism for failed updates is described. The present work provides a complete prototype implementation including bootloader, dual-bank management, and rollback, addressing the gap between the high-level security design of Wang et al. and practical embedded implementation.

This section reviewed published works addressing firmware over-the-air update systems for embedded controllers. The reviewed works collectively establish the motivation for automotive ECU OTA, the advantages of dual-bank flash architectures, the viability of bridge-based wireless delivery, and the security requirements for production deployment. The following section describes the system architecture and design decisions made to address this gap.

3. Implementation Details

3.1.Methodology

3.1.1. Flash memory map of STM32H753ZI

Table 1 Flash memory map of STM32H753ZI

SWAP_BANK = 0	SWAP_BANK = 1	Address	Size	Name
Bank 1	Bank 2	0x08000000- 0x0801FFFF	128KB	Bootloader
		0x08020000- ... 0x080DFFFF	768KB	Firmware region A
		0x080E0000- 0x080FFFFFF	128KB	Metadata region A
Bank 2	Bank 1	0x08100000- 0x0811FFFF	128KB	Bootloader
		0x08120000- ... 0x081DFFFF	768KB	Firmware region B

		0x081E0000- 0x081FFFFFF	128KB	Metadata region B
--	--	----------------------------	-------	-------------------

Table 1 presents the flash memory map of the STM32H753ZI used in this project. During production, the bootloader is programmed into both flash banks to ensure reliability. Additionally, the initial production firmware, along with its associated metadata, is programmed into the primary active bank. During an Over-the-Air (OTA) update, the firmware and metadata in the inactive bank are updated accordingly.

3.1.2. Custom Bootloader

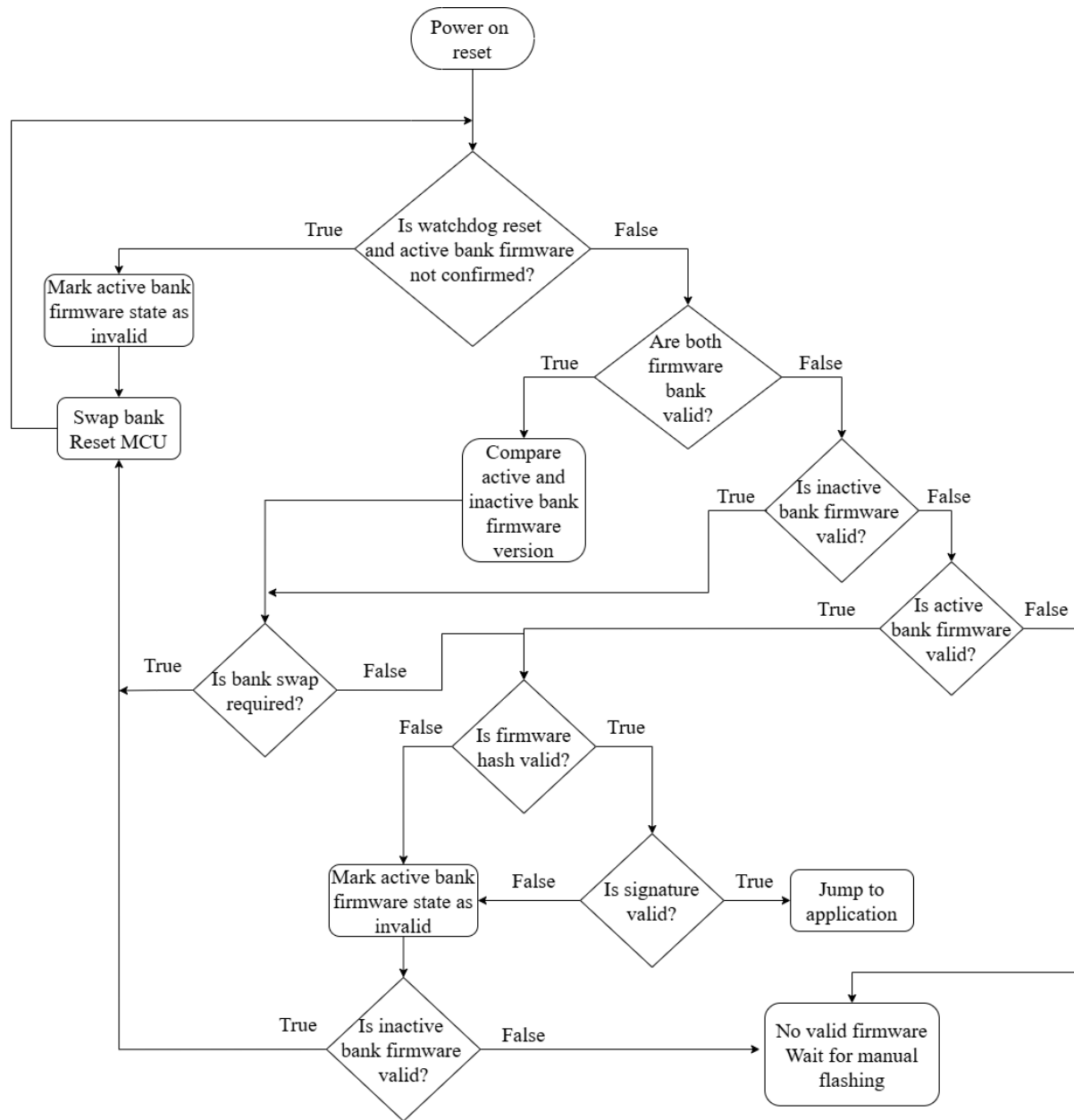


Figure 1 Custom bootloader flowchart

Figure 1 represents custom bootloader which is developed and programmed into Sector 0 of both Region A and Region B of the STM32H753ZI flash memory. The bootloader is the first code executed on every power-on or reset event, running before the main application. It is

implemented as a bare-metal application without an RTOS to minimize boot latency and reduce complexity in the most safety-critical component of the system. The bootloader occupies 128 KB of flash memory in both banks, ensuring it executes correctly regardless of the SWAP_BANK option byte state.

Bootloader reads firmware metadata from both Region A and Region B to determine the validity and version of each firmware image. It applies a priority-ordered boot decision algorithm to select the appropriate firmware region. If a bank swap is required, it programs the SWAP_BANK option byte using the STM32H753ZI Flash Option Byte mechanism, which triggers an automatic system reset and remaps the physical banks.

When a watchdog reset is detected and the active firmware has not been confirmed as valid, the rollback condition is activated. The bootloader first marks the active firmware as invalid to prevent it from being booted again. It then checks whether the inactive region contains valid firmware. If valid backup firmware exists, the bootloader triggers a bank swap by programming the SWAP_BANK option byte, which resets the MCU and boots the previously confirmed firmware. If the inactive region does not contain valid firmware, the bootloader enters a halt state and waits for physical reflash via the ST-LINK debug interface.

Before transferring execution to the selected firmware, the bootloader performs a two-stage cryptographic secure boot verification to ensure that only authenticated and unmodified firmware is permitted to execute on the ECU. In the first stage, the bootloader computes a SHA-256 digest over the complete firmware image residing in the active flash bank using the

STM32H753ZI hardware HASH peripheral. The computed digest is compared against the expected SHA-256 hash stored in the firmware metadata. Hardware acceleration via the dedicated HASH peripheral is employed to minimise boot latency. If the computed digest does not match the stored value, the firmware is considered corrupted, the current bank is marked invalid, and the bootloader attempts to boot from the inactive bank if valid firmware is available, or enters a halt state if no trusted firmware exists.

In the second stage, the bootloader performs ECDSA P-256 digital signature verification using the mbedTLS software library to authenticate the firmware. The verification is performed over the SHA-256 digest computed in the first stage. If signature verification fails, the same recovery procedure as the SHA-256 failure path is executed. Since the STM32H753ZI CRYPT peripheral does not support elliptic curve operations, ECDSA verification is implemented entirely in software. Both the SHA-256 and ECDSA verification times are independently measured and reported over UART at each boot, providing quantitative data for performance evaluation. Only upon successful completion of both verification stages does the bootloader validate the firmware entry point and transfer execution to the selected firmware.

3.1.3. Packet Format

A custom binary communication protocol is designed for reliable firmware transfer between the ESP32 and the STM32H753ZI over a UART interface operating at 115200 bps (bits per second) with 8 data bits, no parity, and one stop bit (8N1). The protocol is designed around a packet-based framing structure with per-packet integrity checking, sequence number for order verification, and an acknowledgement mechanism for reliable delivery. The design follows a

stop-and-wait ARQ (Automatic Repeat Request) approach, where the transmitter sends one packet and waits for an acknowledgement before proceeding to the next.

A custom protocol is chosen over existing standards such as XMODEM or YMODEM for the following reasons. XMODEM does not support firmware header transmission, version negotiation, or flexible payload sizing. YMODEM, while supporting batch transfers, does not provide a mechanism to carry cryptographic metadata such as SHA-256 hash in a structured header packet. The custom protocol is designed specifically to carry all metadata required for secure and reliable OTA firmware update in a single structured start packet.

Table 2 Packet Structure

Header	Type	Sequence number	Length	Data	CRC-16
2 bytes	1 byte	2 bytes	2 bytes	0-256 bytes	2 bytes

Every packet in the protocol follows a fixed-format frame as shown in Table 2. Each frame consists of six fields:

- 2 bytes header 0xAA55 indicates the start of each packet.
- 1 byte type indicates the type of packet.
 - Type = 0x00 indicates handshake packet
 - Type = 0x01 indicates start packet
 - Type = 0x02 indicates data packet
 - Type = 0x03 indicates end packet

- 2 bytes packet sequence number.
 - As mentioned in Table 1, the maximum firmware size which can be supported is 768KB. Thus, sequence number of two-byte length is considered. This helps to detect duplicate packets in cases of missed acknowledgments.
- 2 bytes length field to indicate the length of the data.
- 0-256 bytes of data.
 - In case of start packet, it contains magic number (0xDEADBEEF), firmware version, firmware size, AES nonce, SHA-256 hash and ECDSA signature of the firmware.
 - In case of data packet, it contains data bytes up to 256 bytes. A chunk size of 256 bytes is selected for the firmware transfer protocol. This value is an exact multiple of the STM32H753ZI FLASHWORD size of 32 bytes, eliminating flash alignment complexity, while the resulting 265-byte packet size yields a framing efficiency of 96.6% and a transmission time of approximately 23ms per packet at 115200 bps, ensuring reliable DMA IDLE interrupt detection between consecutive packets.
 - In case of handshake and end packet, there is no data bytes.
- Each packet is protected by a CRC-16-CCITT checksum computed over all packet bytes preceding the CRC field. The generator polynomial is 0x1021 with an initial value of 0xFFFF. CRC-16-CCITT is selected over simpler checksums such as CRC-8 for its superior burst error detection capability. CRC-16-CCITT detects all single-bit errors, all double-bit errors, all burst errors of length 16 or less.

Upon receiving a handshake packet, the STM32H753ZI responds with the active firmware version number. For all other packet types, the STM32H753ZI responds with an acknowledgement on success or a negative acknowledgement on failure.

3.1.4. Security Algorithm Selection

The security pipeline implemented in this work combines three cryptographic primitives selected to satisfy the confidentiality, integrity, and authenticity requirements of automotive firmware delivery.

SHA-256 is selected for firmware integrity verification. It produces a fixed 256-bit digest providing 128-bit collision resistance. SHA-256 is computationally efficient for large streaming data blocks making it suitable for both packet-by-packet accumulation during OTA reception and complete image verification during secure boot. Hardware acceleration via the STM32H753ZI dedicated HASH peripheral reduces boot-time verification latency significantly compared to a software implementation. SHA-256 is preferred over MD5 and SHA-1 which are considered cryptographically broken and are explicitly excluded from automotive security applications.

AES-256-CTR is selected for firmware confidentiality. AES-256 provides 256-bit symmetric encryption. Counter mode converts the block cipher into a stream cipher enabling byte-level encryption without padding, which is essential for firmware images that may not align to the 16-byte AES block boundary. CTR mode supports packet-by-packet decryption without requiring the complete ciphertext in RAM, making it suitable for the streaming OTA reception pipeline. A unique random nonce generated per OTA session ensures that the same firmware binary produces different ciphertext in every update campaign, preventing replay attacks. Hardware acceleration via the STM32H753ZI CRYPT peripheral achieves a 6.4x throughput

improvement over the software mbedTLS baseline.

ECDSA P-256 is selected for firmware authenticity verification. The NIST P-256 curve provides 128-bit security equivalent with a compact 32-byte public key, compared to 256 bytes for RSA-2048 providing equivalent security. Since the STM32H753ZI CRYIP peripheral does not support elliptic curve operations, ECDSA verification is implemented in software via the mbedTLS library.

3.2.Design Overview

Figure 2 illustrates the implemented firmware-over-the-air update system.

The local OTA server, consists of two components. The firmware packaging and management component stores the firmware binary files with the target version to be updated. The OTA campaign management component controls the update campaign by enabling and disabling firmware distribution.

The ESP32 microcontroller acts as a wireless bridge between the local server and the STM32H753ZI. It communicates with the server over HTTP using the Wi-Fi interface, periodically polling for new firmware campaigns. The Wi-Fi and HTTP client component handles server connectivity, firmware binary download. The OTA client component manages version comparison, packet transmission, and retry logic.

The STM32H753ZI contains two distinct firmware layers. The application firmware layer, implemented using FreeRTOS, consists of two concurrent tasks. The OTA task operating at low priority handles UART packet reception, firmware integrity verification, flash programming, and metadata management. The application task operating at normal priority runs the primary ECU

simulation logic and periodically refreshes the independent watchdog timer to confirm healthy operation. The custom bootloader layer executes before the application on every power-on or reset event. The boot decision component evaluates firmware metadata from both flash regions and selects the appropriate firmware using a priority-ordered algorithm. The rollback handling component detects failed firmware updates through the watchdog reset flag and automatically restores the previously confirmed firmware. The jump to application arrow in Figure 2 indicates the bootloader transferring execution control to the application firmware after successful boot decision.

Communication between the ESP32 and the STM32H753ZI is performed over a custom binary UART protocol operating at 115200 bps. The bidirectional arrow indicates firmware data flowing from ESP32 to STM32 and acknowledgement responses returning from STM32 to ESP32.

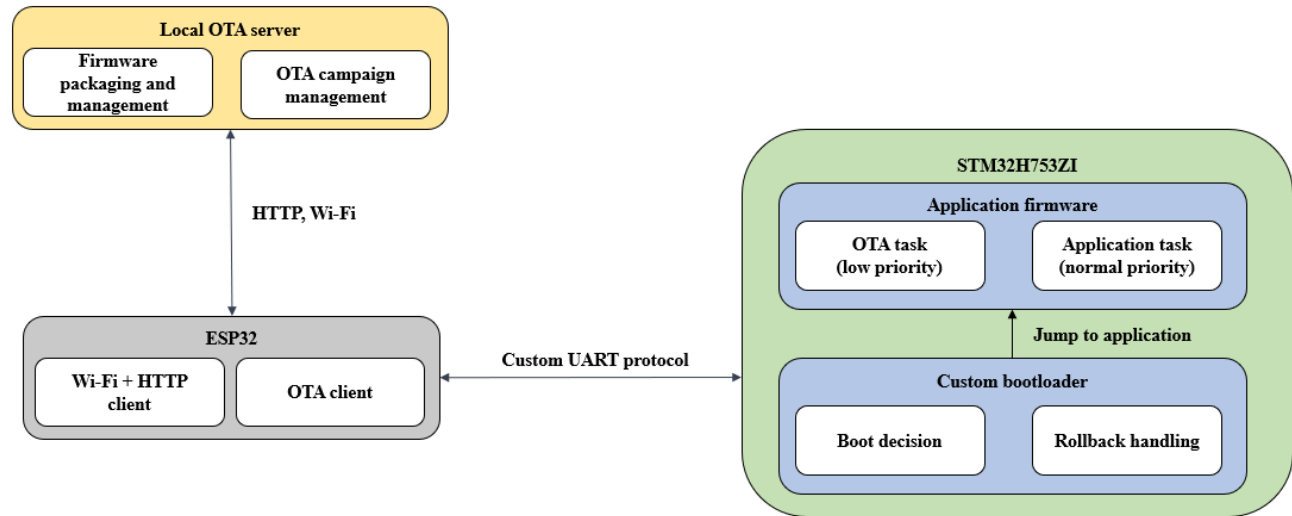


Figure 2 Block diagram of implementation

3.3.Tools and Platform

Hardware Platform:

- STM32H753ZI (Nucleo-H753ZI development board)
 - Core: ARM Cortex-M7
 - Flash: 2MB (dual bank: 1MB per bank)
 - RAM: 1MB
- ESP32 (DevKit)
 - Core: Xtensa LX6 dual-core
 - Flash: 4MB
 - RAM: 520KB

Development Tools:

- IDE's: STM32CubeIDE, Arduino IDE
- Peripheral Configuration: STM32CubeMx
- Debugger/Programmer: ST-LINK/V3
- Programmer: STM32CubeProgrammer

3.4.Assumption or Constraints

Assumptions

- The bootloader binary is identical in both physical flash banks. It is assumed that during production programming, the bootloader is flashed to Sector 0 of both Bank 1 and Bank 2 before the application is deployed.
- Firmware version numbers are monotonically increasing. The bootloader always prefers the higher version number when both regions contain valid firmware. Deliberate firmware downgrade is not supported.
- A power cut between erase and write leaves the metadata sector blank, which the bootloader interprets as invalid firmware.

Constraints

- Maximum firmware size is limited to 768KB.
- The bootloader cannot update itself via OTA.
- Both flash regions must not simultaneously contain invalid firmware. If both regions become invalid then the device requires physical ST-LINK reflash for recovery.

4. Results and Discussion

This chapter presents the results obtained from the implemented OTA firmware update system.

The evaluation covers two aspects:

- (i) Quantitative performance measurements comparing hardware-accelerated and software-only cryptographic implementations for both the secure bootloader and OTA update pipeline.
- (ii) Functional validation through a set of test scenarios designed to verify correct system behavior under normal operation and failure conditions.

All measurements were obtained on the Nucleo-H753ZI development board (STM32H753ZI, ARM Cortex-M7 at 480 MHz) interfaced with an ESP32 DevKit over UART at 115200 bps. The firmware was compiled using STM32CubeIDE with the default optimization settings (-O0). The application firmware executed as a periodic FreeRTOS task with a cycle time of 500 ms, representing a simplified automotive ECU application workload running concurrently with the OTA update task and watchdog servicing operations. The firmware image used for performance measurements was 128 KB in size, transferred in 512 packets of 256 bytes each.

4.1. Hardware setup

Figure 3 presents the hardware setup used for project demonstration. The STM32H753ZI Nucleo board runs the custom bootloader and FreeRTOS application firmware. The ESP32 DevKit acts as the Wi-Fi bridge, downloading firmware from the local OTA server over HTTP and relaying it to the STM32 over a custom binary UART protocol. UART output from both devices is monitored in real time using Tera Term serial terminals. Firmware images are packaged and signed using the

custom firmware packaging tool, which computes the SHA-256 hash, encrypts the payload using AES-256-CTR, and embeds the ECDSA P-256 signature into the firmware header.

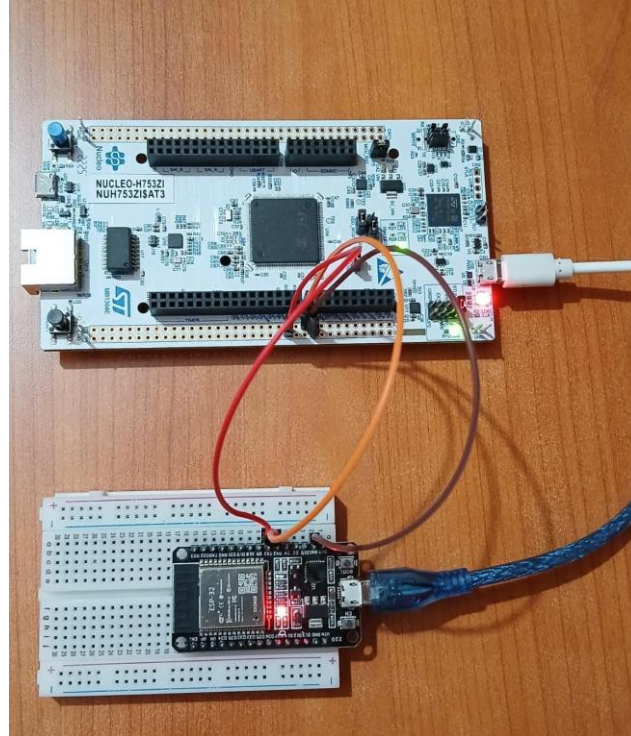


Figure 3 Hardware setup

4.2. Secure bootloader performance

Table 3 Secure bootloader cryptographic performance (firmware size: 128 KB)

Operation	Software only	Hardware accelerated
SHA-256 hash verification	147 ms	9 ms
ECDSA P-256 signature verification	931 ms	946 ms
Total secure boot time	1094 ms	970 ms

Table 3 presents the measured execution times for the two cryptographic operations performed by

the secure bootloader at each power-on or reset event, for a 128 KB firmware image.

The SHA-256 hash verification is performed using the STM32H753ZI hardware HASH peripheral, which offloads the digest computation from the Cortex-M7 core to a dedicated hardware block. The hardware-accelerated SHA-256 completes in 9 ms compared to 147 ms in the software-only baseline, representing a 16.3x speedup. This improvement directly reduces boot latency and is significant in automotive ECU contexts where deterministic boot times are required.

ECDSA P-256 signature verification, is executed entirely in software using the mbedTLS library in both the hardware-accelerated and software-only configurations. This is because the STM32H753ZI CRYPT peripheral supports symmetric cryptographic operations but does not provide hardware acceleration for elliptic curve operations. Both measurements reflect the same mbedTLS software execution path, and the marginal difference of 15 ms between the two runs (931 ms vs 946 ms) lies within normal measurement variance and is not a meaningful performance difference.

The total boot time reduces from 1094 ms (software only) to 970 ms (hardware accelerated), a saving of 124 ms, due to the SHA-256 hardware acceleration. For systems with strict boot-time requirements, this reduction is meaningful. Both boot times were measured to be consistent across repeated runs, confirming the determinism of the bootloader implementation.

The secure boot performance as tabulated in Table 4 for the proposed authentication mechanism scales efficiently with increasing firmware size. Although the firmware image size increased from 128 KB to 768 KB, the total secure boot time increased by only 41 ms. This behaviour occurred because SHA-256 integrity verification time scales with firmware size, whereas ECDSA P-256 signature verification time remains nearly constant due to the fixed-size signature operation. The

results show that ECDSA verification contributed more than 90% of the total secure boot latency, indicating that asymmetric cryptographic authentication forms the dominant computational overhead in the secure boot process.

Table 4 Secure bootloader cryptographic performance for various firmware sizes

Firmware size	SHA-256 hash verification	ECDSA P-256 signature verification	Total secure boot time
128 KB	8ms	943 ms	966 ms
256 KB	16 ms	947 ms	977 ms
512 KB	32 ms	949 ms	997 ms
768 KB	49 ms	943 ms	1007 ms

4.3.OTA update pipeline performance

Table 5 presents the timing breakdown for each stage of the OTA firmware update process, measured on both the STM32 (receiver) and ESP32 (transmitter) sides for a 128 KB firmware image.

Table 5 OTA update timing comparison software vs hardware accelerated (firmware size: 128 KB)

Operation	Software only	Hardware accelerated
Flash erase (7 sectors)	6202 ms	6181 ms
AES 256 decrypt (total)	218624 μ s	34318 μ s
AES 256 decrypt (avg/packet)	427 μ s	67 μ s
Flash write (total)	379391 μ s	367968 μ s

Flash write (avg/packet)	740 μ s	718 μ s
SHA-256 accumulation (total)	147061 μ s	7767 μ s
SHA-256 accumulation (avg/packet)	287 μ s	15 μ s
ECDSA verification	1068 ms	1043 ms
Total OTA time (STM32 side)	20902 ms	20707 ms
Total OTA time (ESP32 side)	20696 ms	20514 ms
Total data packets processed	512	512

The most significant performance gain from hardware acceleration is observed in the AES-256 CTR decryption stage. The hardware-accelerated CRYPT peripheral completes decryption of the full 128 KB firmware payload in 34318 μ s (34.3 ms), compared to 218624 μ s (218.6 ms) in software, which corresponds to a 6.4x speedup. On a per-packet basis, hardware decryption takes 67 μ s versus 427 μ s in software, a 6.4x improvement that is consistent across the 512-packet transfer. This reduction in per-packet processing time is particularly important in the context of the stop-and-wait ARQ protocol used, where per-packet processing directly affects the throughput achievable over the 115200 bps UART link.

SHA-256 accumulation over the received firmware stream shows an even larger hardware advantage: 7767 μ s with hardware versus 147061 μ s in software, which corresponds to a 18.9x speedup. Per packet, this reduces from 287 μ s to 15 μ s. The combined effect of hardware AES decryption and SHA-256 accumulation reduces the total cryptographic processing overhead from approximately 366 ms to 42 ms for the 128 KB payload.

Flash erase and flash write times show minimal difference between hardware and software configurations as these operations are constrained by the STM32H753ZI flash controller timing and are identical regardless of the cryptographic configuration. Flash erase of 7 sectors takes approximately 6.2 seconds in both cases, and this dominates the total OTA time.

ECDSA verification at the end of the OTA transfer takes approximately similar time as it is executed entirely in software using the mbedTLS library.

The total OTA time on the STM32 side is 20902 ms (software) and 20707 ms (hardware), a reduction of approximately 195 ms. The total OTA time on the ESP32 side is 20696 ms (software) and 20514 ms (hardware), a difference of 182 ms. The relatively lower reduction in total OTA time despite significant cryptographic speedups is due to the dominance of the flash erase time (~6.2 s) and the UART transfer time, both of which are unaffected by cryptographic acceleration. The practical significance of hardware acceleration would increase substantially for larger firmware images where the proportional contribution of cryptographic processing to total OTA time is higher.

4.4.Functional test results

The system was validated through functional test scenarios covering normal operation, security rejection, and failure recovery cases. Each scenario was tested by configuring the firmware packaging tool appropriately (corrupting the SHA-256 hash, ECDSA signature, or AES nonce as required) and observing the UART output from both the STM32 and ESP32. In the results below, a firmware version value of 4294967295 (0xFFFFFFFF) indicates an uninitialized or erased flash region. Below are the different test scenarios which are tested.

- 1) No valid firmware in each bank.

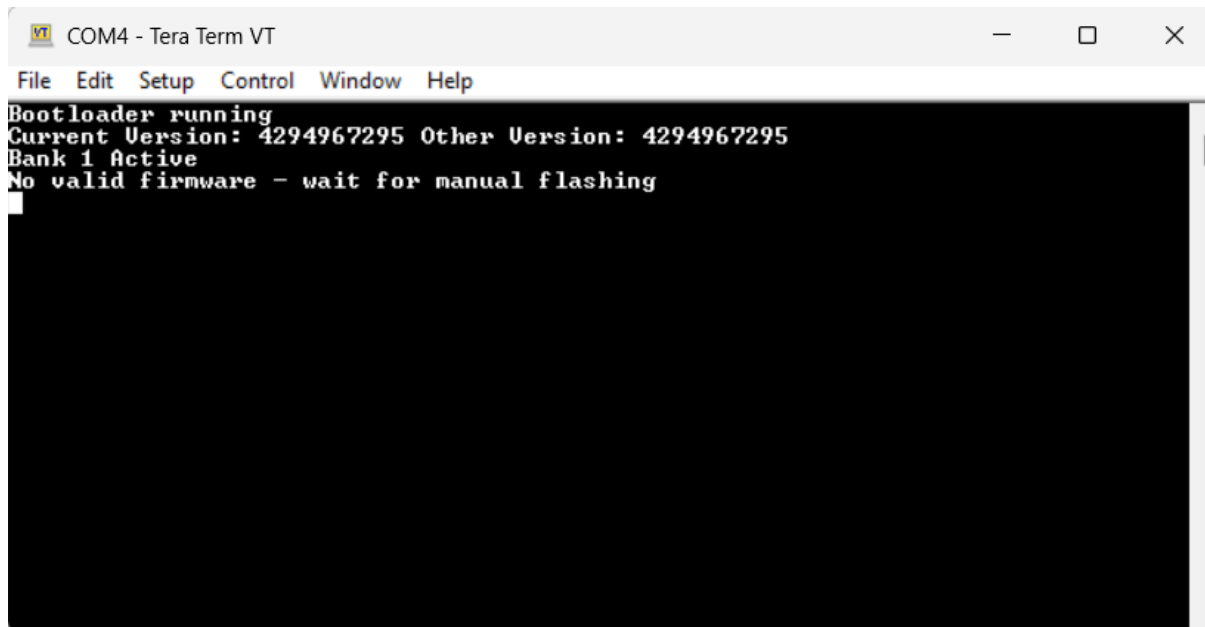


Figure 4 Test case 1: No valid firmware in each bank

Bootloader is programmed in both memory banks, while no valid firmware is present in either bank. In this scenario, as shown in Figure 4, bootloader remains in a waiting state until a valid firmware image is manually flashed.

- 2) Valid firmware in Bank 1 and Bank 2 is empty.

Figure 5 illustrates the scenario when valid firmware is present in bank 1, while bank 2 does not contain any firmware.

```

COM4 - Tera Term VT
File Edit Setup Control Window Help
Bootloader running
Current Version: 1 Other Version: 4294967295
Bank 1 Active
SecureBoot: Starting...
SecureBoot: SHA256 PASSED
SecureBoot: ECDSA PASSED
SecureBoot: Firmware TRUSTED

=== BOOT TIMING ===
Firmware size : 100788 bytes
SHA256: 7ms
ECDSA: 941ms
Total: 963ms

SecureBoot: Jumping to application
Application1 Bank1 Running
[00000] #1 : App1-Bank1 : Spd=79.5km/h : Pres=0.4bar : ABS=off : Period=0ms
[00500] #2 : App1-Bank1 : Spd=79.0km/h : Pres=0.8bar : ABS=off : Period=500ms
[01000] #3 : App1-Bank1 : Spd=78.5km/h : Pres=1.2bar : ABS=off : Period=500ms
[01500] #4 : App1-Bank1 : Spd=78.0km/h : Pres=1.6bar : ABS=off : Period=500ms
[02000] #5 : App1-Bank1 : Spd=77.5km/h : Pres=2.0bar : ABS=off : Period=500ms
[02500] #6 : App1-Bank1 : Spd=77.0km/h : Pres=2.4bar : ABS=off : Period=500ms
[03000] #7 : App1-Bank1 : Spd=76.5km/h : Pres=2.7bar : ABS=off : Period=500ms

```

Figure 5 Test case 2: Valid firmware in Bank 1 and Bank 2 is empty

- 3) Normal OTA update: Bank 1 active, new firmware received.

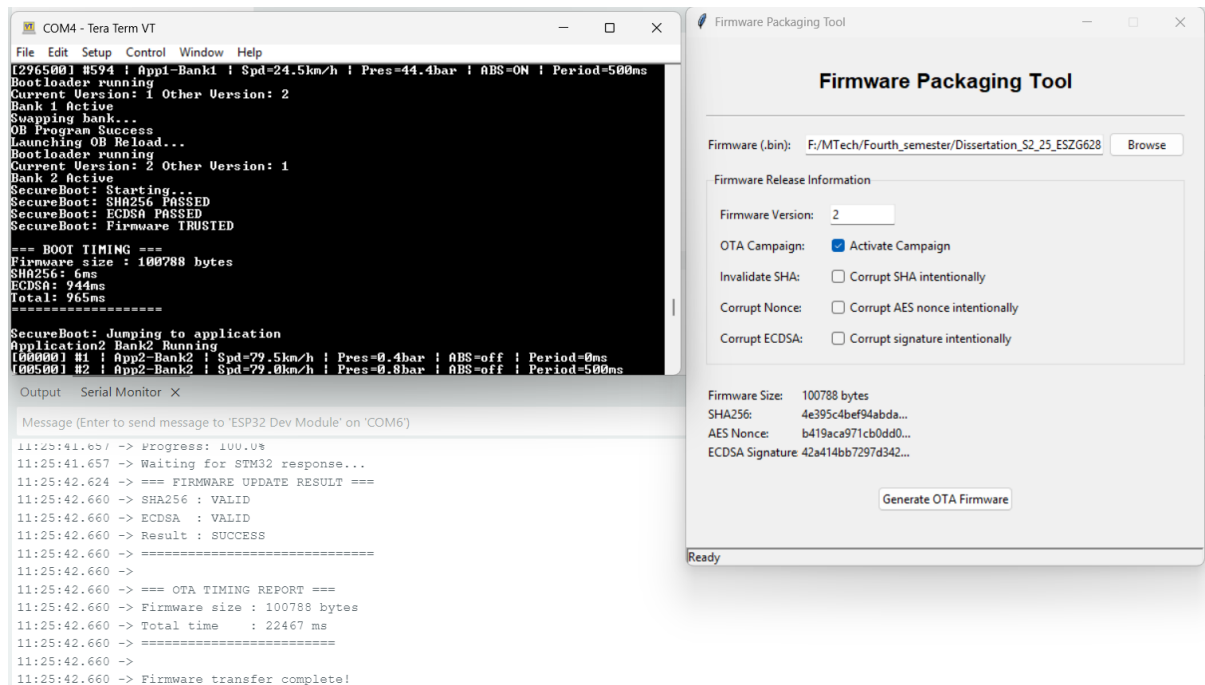


Figure 6 Test case 3: Normal OTA update: Bank 1 active, new firmware received

Figure 6 depicts the case of normal OTA update process. When the firmware in bank 1 is

running, an OTA update is received. The new firmware is transferred from the ESP32 and written to the inactive bank of the STM32H753ZI. When there is a reset triggered, the bank swap occurs and the newly received firmware begins execution.

4) OTA update with SHA-256 hash corrupted.

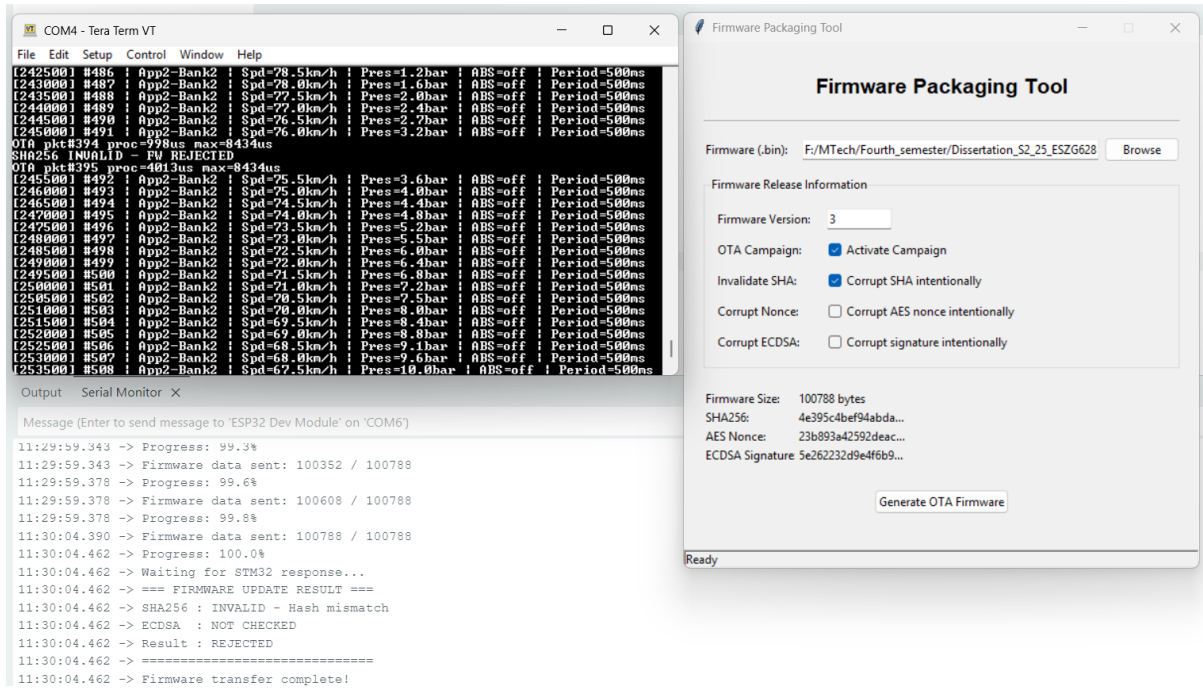


Figure 7 Test case 4a: OTA update with SHA-256 hash corrupted

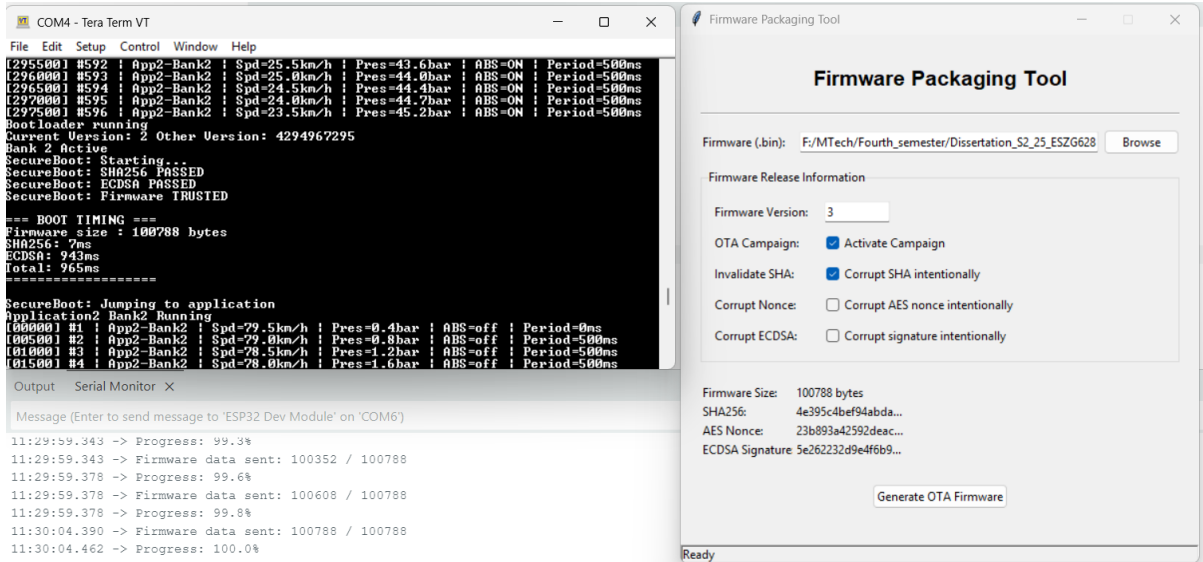


Figure 8 Test case 4b: OTA update with SHA-256 hash corrupted

Figure 7 and Figure 8 illustrate the scenario of an OTA update with a SHA-256 hash mismatch. The received firmware fails the integrity check. Thus, when a reset detected the previous running valid firmware continues to execute.

5) OTA update with ECDSA signature corrupted.

Figure 9 and Figure 10 illustrate the scenario of an OTA update with ECDSA signature verification failure. The computed signature based on the received firmware does not match the received signature. Thus, when a reset detected the previous running valid firmware continues to execute.

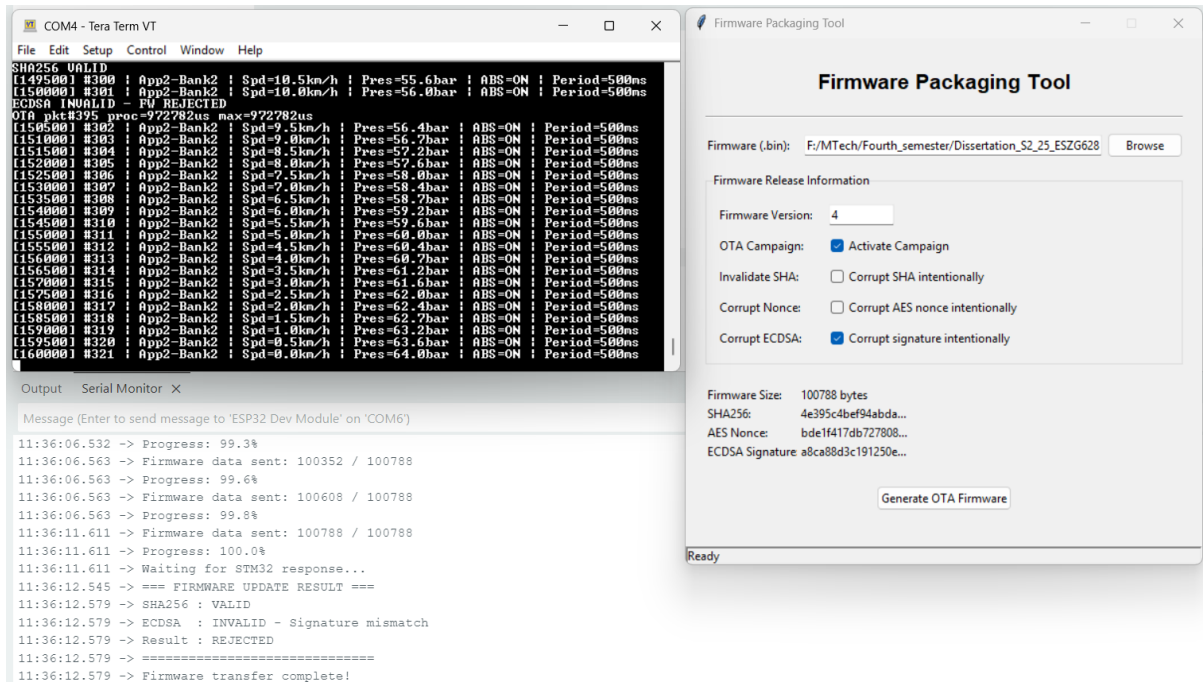


Figure 9 Test case 5a: OTA update with ECDSA signature corrupted

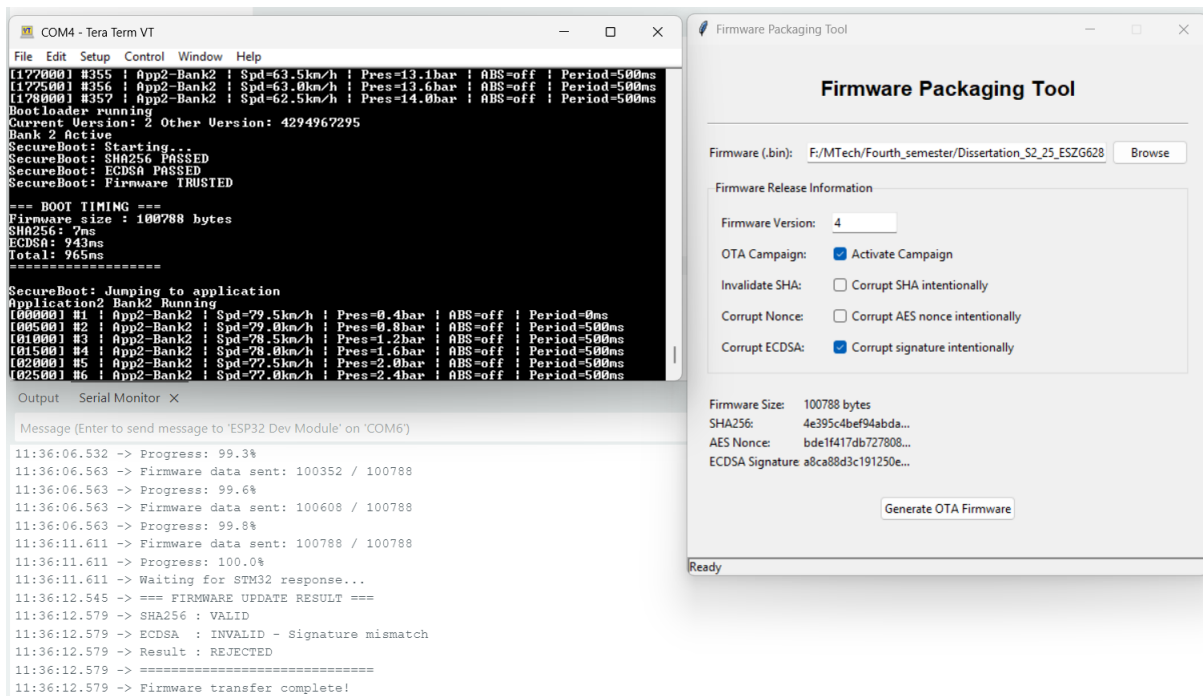


Figure 10 Test case 5b: OTA update with ECDSA signature corrupted

6) OTA update with wrong nonce for AES decryption.

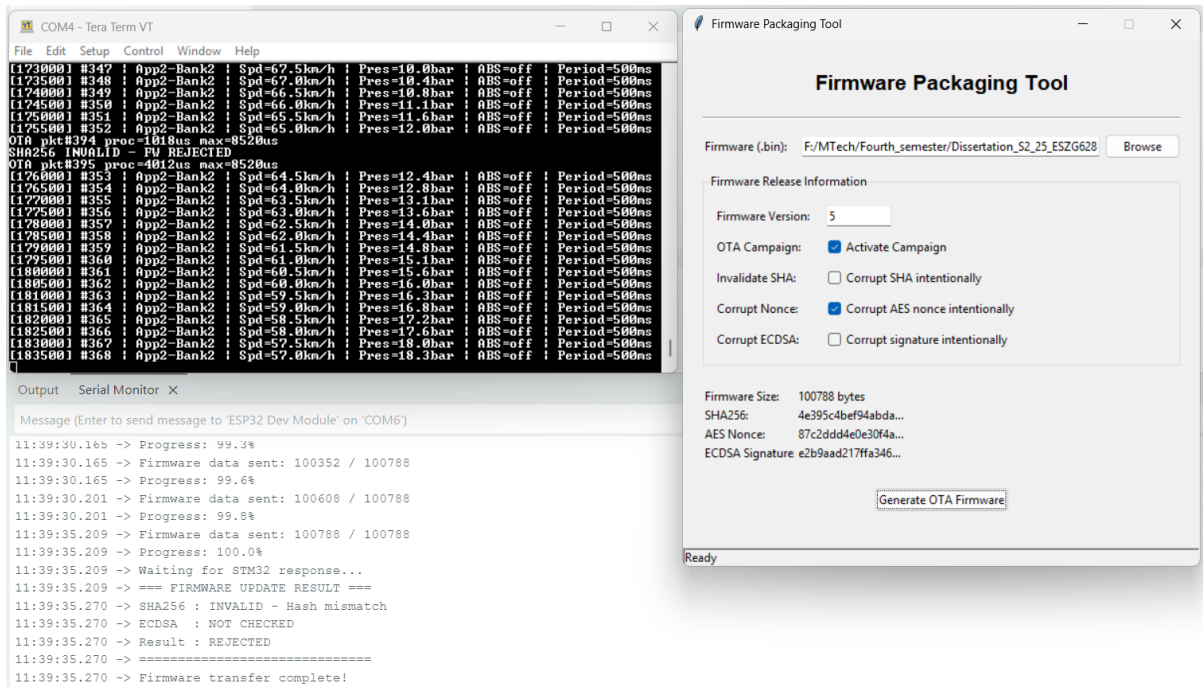


Figure 11 Test case 6a: OTA update with wrong nonce for AES decryption

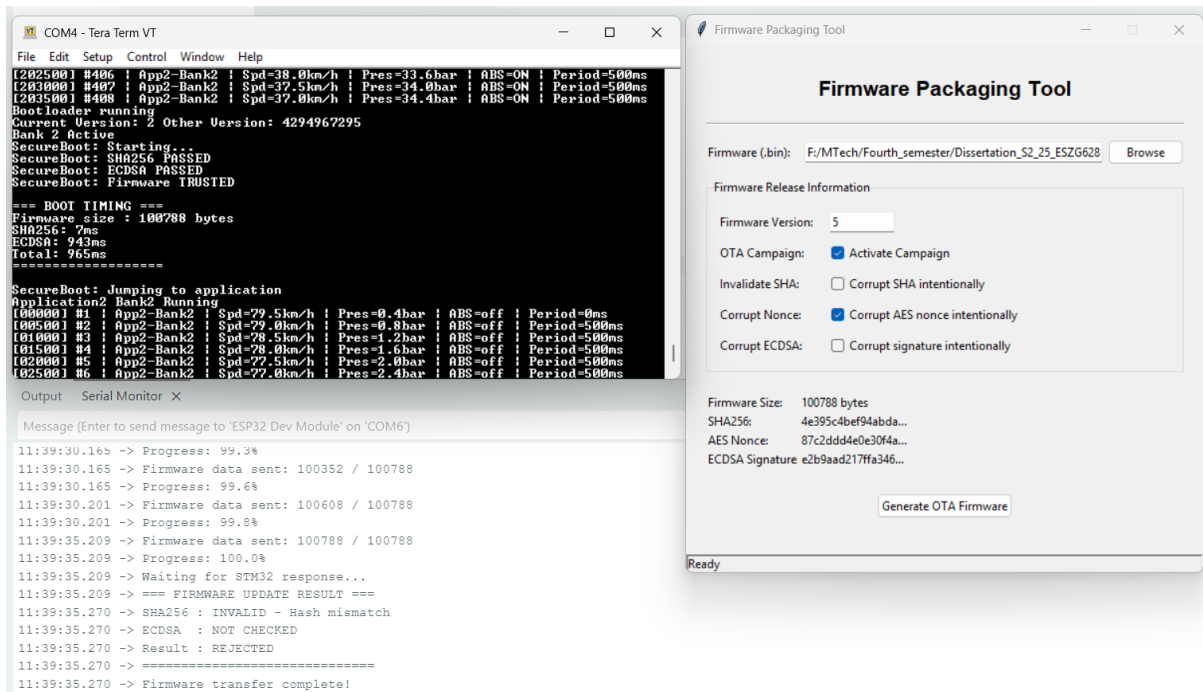


Figure 12 Test case 6b: OTA update with wrong nonce for AES decryption

Figure 11 and Figure 12 illustrate the scenario of an OTA update with wrong AES nonce value received for decryption. Incorrect nonce which is used for decryption produces invalid firmware data. The SHA-256 hash computed over the decrypted firmware does not match the expected hash value. Thus, there is integrity verification failure and when a reset detected the previous running valid firmware continues to execute.

7) Valid OTA update but updated firmware triggers watchdog reset.

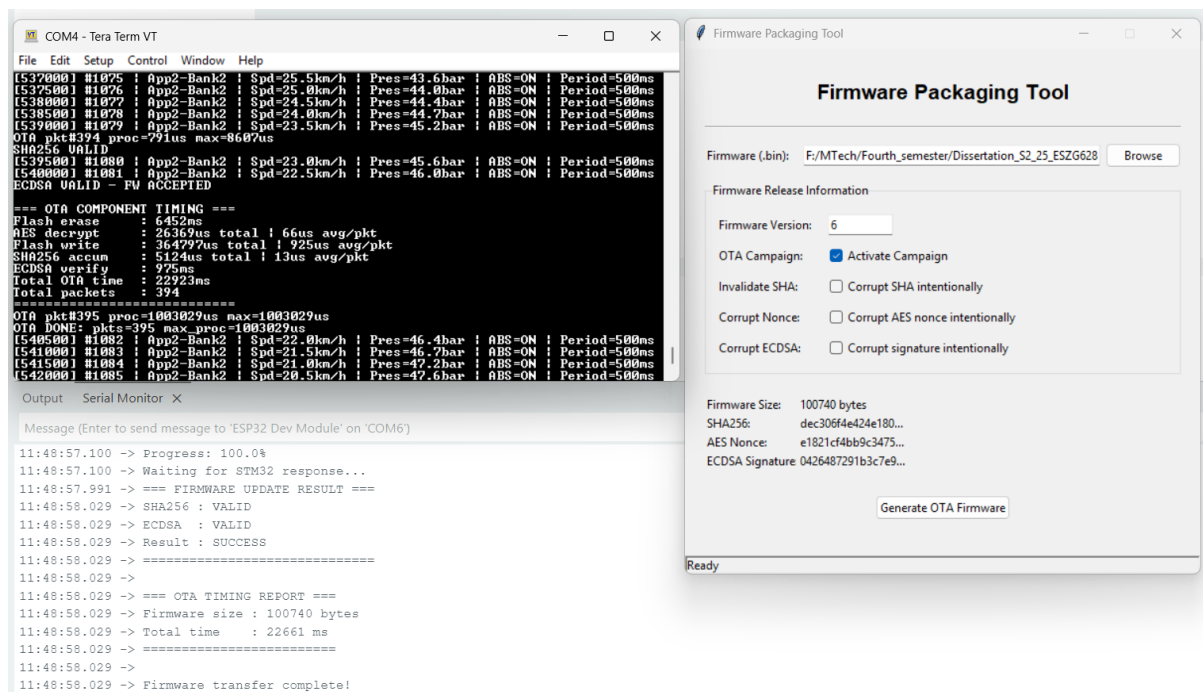


Figure 13 Test case 7a: Valid OTA update but updated firmware triggers watchdog reset

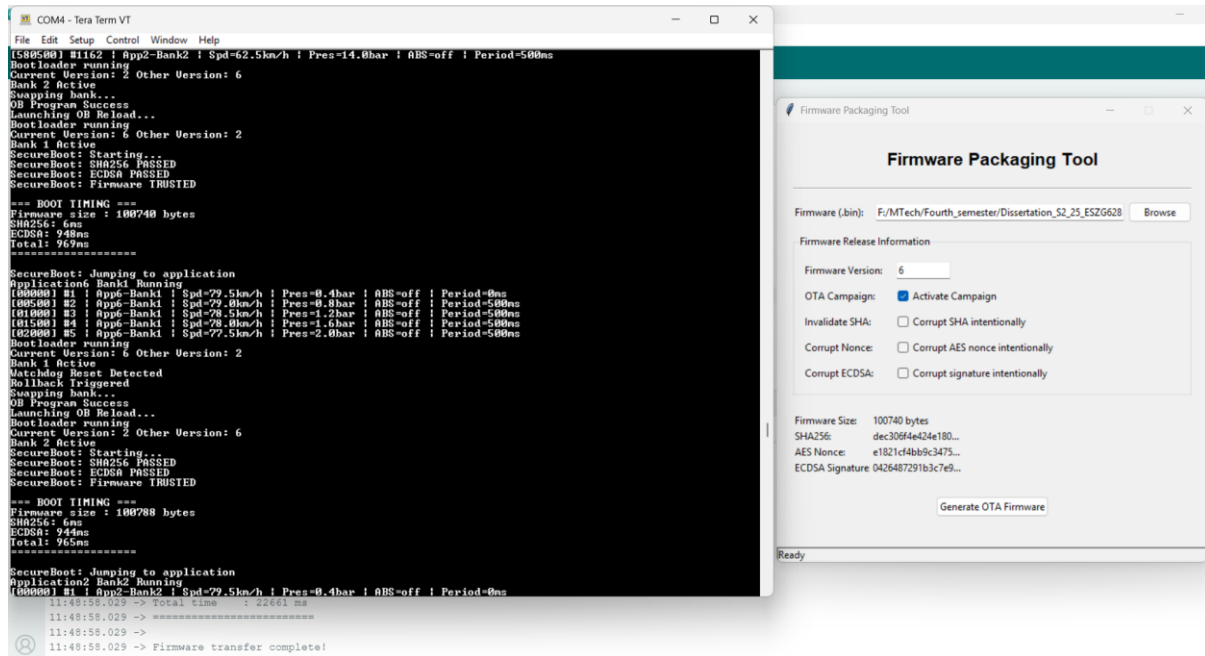


Figure 14 Test case 7b: Valid OTA update but updated firmware triggers watchdog reset

Figure 13 and Figure 14 illustrate the scenario where a valid OTA firmware update is applied, but the updated firmware becomes unresponsive and triggers a watchdog reset. In this case as the received firmware is not reliable, rollback is triggered to the previous firmware.

8) Firmware transfer interrupted.

Figure 15 illustrates the scenario where a firmware transfer is interrupted. In this case due to incomplete firmware transfer, the previous firmware continues execution.

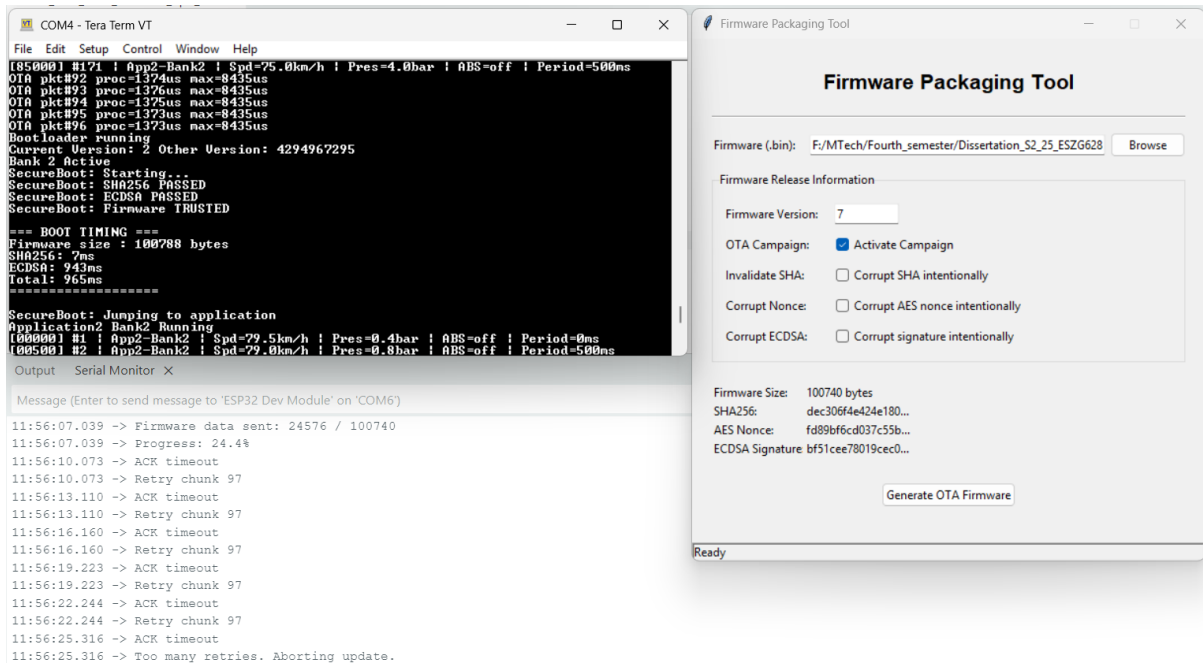


Figure 15 Test case 8: Firmware transfer interrupted

9) Boot failed due to invalid SHA-256.

a) Both banks valid firmware

Initially, both bank contains valid firmware as shown in Figure 16. Firmware in active bank is modified. During bootup when the hash of the active bank is checked there is mismatch. As active bank integrity checks failed, bank is switched and inactive bank firmware executes as shown in Figure 17.

```
COM4 - Tera Term VT
File Edit Setup Control Window Help
Bootloader running
Current Version: 9 Other Version: 8
Bank 2 Active
SecureBoot: Starting...
SecureBoot: SHA256 PASSED
SecureBoot: ECDSA PASSED
SecureBoot: Firmware TRUSTED

=== BOOT TIMING ===
Firmware size : 100788 bytes
SHA256: 6ms
ECDSA: 949ms
Total: 970ms

SecureBoot: Jumping to application
Application9 Bank2 Running
[00000] #1 : App9-Bank2 : Spd=79.5km/h : Pres=0.4bar : ABS=off : Period=0ms
[00500] #2 : App9-Bank2 : Spd=79.0km/h : Pres=0.8bar : ABS=off : Period=500ms
[01000] #3 : App9-Bank2 : Spd=78.5km/h : Pres=1.2bar : ABS=off : Period=500ms
[01500] #4 : App9-Bank2 : Spd=78.0km/h : Pres=1.6bar : ABS=off : Period=500ms
[02000] #5 : App9-Bank2 : Spd=77.5km/h : Pres=2.0bar : ABS=off : Period=500ms
[02500] #6 : App9-Bank2 : Spd=77.0km/h : Pres=2.4bar : ABS=off : Period=500ms
[03000] #7 : App9-Bank2 : Spd=76.5km/h : Pres=2.7bar : ABS=off : Period=500ms
[03500] #8 : App9-Bank2 : Spd=76.0km/h : Pres=3.2bar : ABS=off : Period=500ms
[04000] #9 : App9-Bank2 : Spd=75.5km/h : Pres=3.6bar : ABS=off : Period=500ms
[04500] #10 : App9-Bank2 : Spd=75.0km/h : Pres=4.0bar : ABS=off : Period=500ms
[05000] #11 : App9-Bank2 : Spd=74.5km/h : Pres=4.4bar : ABS=off : Period=500ms
[05500] #12 : App9-Bank2 : Spd=74.0km/h : Pres=4.8bar : ABS=off : Period=500ms
[06000] #13 : App9-Bank2 : Spd=73.5km/h : Pres=5.2bar : ABS=off : Period=500ms
[06500] #14 : App9-Bank2 : Spd=73.0km/h : Pres=5.5bar : ABS=off : Period=500ms
[07000] #15 : App9-Bank2 : Spd=72.5km/h : Pres=6.0bar : ABS=off : Period=500ms
[07500] #16 : App9-Bank2 : Spd=72.0km/h : Pres=6.4bar : ABS=off : Period=500ms
[08000] #17 : App9-Bank2 : Spd=71.5km/h : Pres=6.8bar : ABS=off : Period=500ms
[08500] #18 : App9-Bank2 : Spd=71.0km/h : Pres=7.2bar : ABS=off : Period=500ms
[09000] #19 : App9-Bank2 : Spd=70.5km/h : Pres=7.5bar : ABS=off : Period=500ms
[09500] #20 : App9-Bank2 : Spd=70.0km/h : Pres=8.0bar : ABS=off : Period=500ms
[10000] #21 : App9-Bank2 : Spd=69.5km/h : Pres=8.4bar : ABS=off : Period=500ms
[10500] #22 : App9-Bank2 : Spd=69.0km/h : Pres=8.8bar : ABS=off : Period=500ms
[11000] #23 : App9-Bank2 : Spd=68.5km/h : Pres=9.1bar : ABS=off : Period=500ms
[11500] #24 : App9-Bank2 : Spd=68.0km/h : Pres=9.6bar : ABS=off : Period=500ms
[12000] #25 : App9-Bank2 : Spd=67.5km/h : Pres=10.0bar : ABS=off : Period=500ms
[12500] #26 : App9-Bank2 : Spd=67.0km/h : Pres=10.4bar : ABS=off : Period=500ms
[13000] #27 : App9-Bank2 : Spd=66.5km/h : Pres=10.8bar : ABS=off : Period=500ms
```

Figure 16 Test case 9a(i): Secure boot integrity verification failed due to invalid SHA-256 hash

```
COM4 - Tera Term VT
File Edit Setup Control Window Help
[43500] #88 : App9-Bank2 : Spd=36.0km/h : Pres=35.2bar : ABS=ON : Period=500ms
[44000] #89 : App9-Bank2 : Spd=35.5km/h : Pres=35.6bar : ABS=ON : Period=500ms
[44500] #90 : App9-Bank2 : Spd=35.0km/h : Pres=36.0bar : ABS=ON : Period=500ms
Bootloader running
Current Version: 9 Other Version: 8
Bank 2 Active
SecureBoot: Starting...
SecureBoot: SHA256 FAILED
SecureBoot: BOOT BLOCKED
SecureBoot: Trying other bank
OB Program Success
Launching OB Reload...
Bootloader running
Current Version: 8 Other Version: 9
Bank 1 Active
SecureBoot: Starting...
SecureBoot: SHA256 PASSED
SecureBoot: ECDSA PASSED
SecureBoot: Firmware TRUSTED

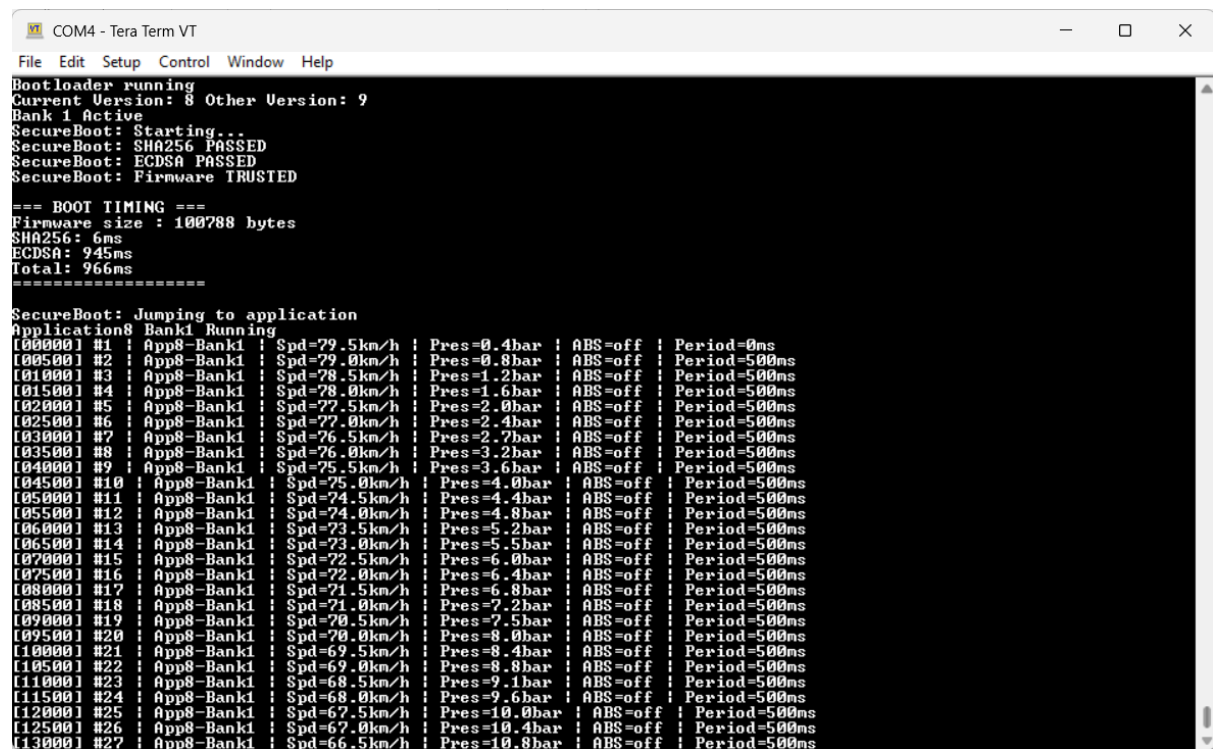
=== BOOT TIMING ===
Firmware size : 100788 bytes
SHA256: 6ms
ECDSA: 945ms
Total: 966ms

SecureBoot: Jumping to application
Application8 Bank1 Running
[00000] #1 : App8-Bank1 : Spd=79.5km/h : Pres=0.4bar : ABS=off : Period=0ms
[00500] #2 : App8-Bank1 : Spd=79.0km/h : Pres=0.8bar : ABS=off : Period=500ms
[01000] #3 : App8-Bank1 : Spd=78.5km/h : Pres=1.2bar : ABS=off : Period=500ms
[01500] #4 : App8-Bank1 : Spd=78.0km/h : Pres=1.6bar : ABS=off : Period=500ms
[02000] #5 : App8-Bank1 : Spd=77.5km/h : Pres=2.0bar : ABS=off : Period=500ms
[02500] #6 : App8-Bank1 : Spd=77.0km/h : Pres=2.4bar : ABS=off : Period=500ms
[03000] #7 : App8-Bank1 : Spd=76.5km/h : Pres=2.7bar : ABS=off : Period=500ms
[03500] #8 : App8-Bank1 : Spd=76.0km/h : Pres=3.2bar : ABS=off : Period=500ms
[04000] #9 : App8-Bank1 : Spd=75.5km/h : Pres=3.6bar : ABS=off : Period=500ms
[04500] #10 : App8-Bank1 : Spd=75.0km/h : Pres=4.0bar : ABS=off : Period=500ms
[05000] #11 : App8-Bank1 : Spd=74.5km/h : Pres=4.4bar : ABS=off : Period=500ms
[05500] #12 : App8-Bank1 : Spd=74.0km/h : Pres=4.8bar : ABS=off : Period=500ms
[06000] #13 : App8-Bank1 : Spd=73.5km/h : Pres=5.2bar : ABS=off : Period=500ms
[06500] #14 : App8-Bank1 : Spd=73.0km/h : Pres=5.5bar : ABS=off : Period=500ms
[07000] #15 : App8-Bank1 : Spd=72.5km/h : Pres=6.0bar : ABS=off : Period=500ms
```

Figure 17 Test case 9a(ii): Secure boot integrity verification failed due to invalid SHA-256 hash

b) Single bank valid firmware

Initially only active bank contains valid firmware as shown in Figure 18. Firmware in active bank is modified. During bootup when the hash of the active bank is checked there is mismatch. Active bank integrity check failed and inactive bank does not contain valid firmware. As there are no valid firmware, bootloader remains in a waiting state until a valid firmware image is manually flashed as shown in Figure 19.



```
COM4 - Tera Term VT
File Edit Setup Control Window Help
Bootloader running
Current Version: 8 Other Version: 9
Bank 1 Active
SecureBoot: Starting...
SecureBoot: SHA256 PASSED
SecureBoot: ECDSA PASSED
SecureBoot: Firmware TRUSTED

=== BOOT TIMING ===
Firmware size : 100788 bytes
SHA256: 6ms
ECDSA: 945ms
Total: 966ms
=====
SecureBoot: Jumping to application
Application8 Bank1 Running
[00000] #1 : App8-Bank1 : Spd=79.5km/h : Pres=0.4bar : ABS=off : Period=0ms
[00500] #2 : App8-Bank1 : Spd=79.0km/h : Pres=0.8bar : ABS=off : Period=500ms
[01000] #3 : App8-Bank1 : Spd=78.5km/h : Pres=1.2bar : ABS=off : Period=500ms
[01500] #4 : App8-Bank1 : Spd=78.0km/h : Pres=1.6bar : ABS=off : Period=500ms
[02000] #5 : App8-Bank1 : Spd=77.5km/h : Pres=2.0bar : ABS=off : Period=500ms
[02500] #6 : App8-Bank1 : Spd=77.0km/h : Pres=2.4bar : ABS=off : Period=500ms
[03000] #7 : App8-Bank1 : Spd=76.5km/h : Pres=2.7bar : ABS=off : Period=500ms
[03500] #8 : App8-Bank1 : Spd=76.0km/h : Pres=3.2bar : ABS=off : Period=500ms
[04000] #9 : App8-Bank1 : Spd=75.5km/h : Pres=3.6bar : ABS=off : Period=500ms
[04500] #10 : App8-Bank1 : Spd=75.0km/h : Pres=4.0bar : ABS=off : Period=500ms
[05000] #11 : App8-Bank1 : Spd=74.5km/h : Pres=4.4bar : ABS=off : Period=500ms
[05500] #12 : App8-Bank1 : Spd=74.0km/h : Pres=4.8bar : ABS=off : Period=500ms
[06000] #13 : App8-Bank1 : Spd=73.5km/h : Pres=5.2bar : ABS=off : Period=500ms
[06500] #14 : App8-Bank1 : Spd=73.0km/h : Pres=5.5bar : ABS=off : Period=500ms
[07000] #15 : App8-Bank1 : Spd=72.5km/h : Pres=6.0bar : ABS=off : Period=500ms
[07500] #16 : App8-Bank1 : Spd=72.0km/h : Pres=6.4bar : ABS=off : Period=500ms
[08000] #17 : App8-Bank1 : Spd=71.5km/h : Pres=6.8bar : ABS=off : Period=500ms
[08500] #18 : App8-Bank1 : Spd=71.0km/h : Pres=7.2bar : ABS=off : Period=500ms
[09000] #19 : App8-Bank1 : Spd=70.5km/h : Pres=7.5bar : ABS=off : Period=500ms
[09500] #20 : App8-Bank1 : Spd=70.0km/h : Pres=8.0bar : ABS=off : Period=500ms
[10000] #21 : App8-Bank1 : Spd=69.5km/h : Pres=8.4bar : ABS=off : Period=500ms
[10500] #22 : App8-Bank1 : Spd=69.0km/h : Pres=8.8bar : ABS=off : Period=500ms
[11000] #23 : App8-Bank1 : Spd=68.5km/h : Pres=9.1bar : ABS=off : Period=500ms
[11500] #24 : App8-Bank1 : Spd=68.0km/h : Pres=9.6bar : ABS=off : Period=500ms
[12000] #25 : App8-Bank1 : Spd=67.5km/h : Pres=10.0bar : ABS=off : Period=500ms
[12500] #26 : App8-Bank1 : Spd=67.0km/h : Pres=10.4bar : ABS=off : Period=500ms
[13000] #27 : App8-Bank1 : Spd=66.5km/h : Pres=10.8bar : ABS=off : Period=500ms
```

Figure 18 Test case 9b(i): Secure boot integrity verification failed due to invalid SHA-256 hash

```

COM4 - Tera Term VT
File Edit Setup Control Window Help
[31000] #63 : App8-Bank1 : Spd=48.5km/h : Pres=25.2bar : ABS=off : Period=500ms
[31500] #64 : App8-Bank1 : Spd=48.0km/h : Pres=25.6bar : ABS=off : Period=500ms
[32000] #65 : App8-Bank1 : Spd=47.5km/h : Pres=26.0bar : ABS=off : Period=500ms
[32500] #66 : App8-Bank1 : Spd=47.0km/h : Pres=26.3bar : ABS=off : Period=500ms
[33000] #67 : App8-Bank1 : Spd=46.5km/h : Pres=26.8bar : ABS=off : Period=500ms
[33500] #68 : App8-Bank1 : Spd=46.0km/h : Pres=27.2bar : ABS=off : Period=500ms
[34000] #69 : App8-Bank1 : Spd=45.5km/h : Pres=27.6bar : ABS=off : Period=500ms
[34500] #70 : App8-Bank1 : Spd=45.0km/h : Pres=28.0bar : ABS=off : Period=500ms
[35000] #71 : App8-Bank1 : Spd=44.5km/h : Pres=28.3bar : ABS=off : Period=500ms
[35500] #72 : App8-Bank1 : Spd=44.0km/h : Pres=28.8bar : ABS=off : Period=500ms
[36000] #73 : App8-Bank1 : Spd=43.5km/h : Pres=29.2bar : ABS=off : Period=500ms
[36500] #74 : App8-Bank1 : Spd=43.0km/h : Pres=29.6bar : ABS=off : Period=500ms
[37000] #75 : App8-Bank1 : Spd=42.5km/h : Pres=30.0bar : ABS=off : Period=500ms
[37500] #76 : App8-Bank1 : Spd=42.0km/h : Pres=30.3bar : ABS=ON : Period=500ms
[38000] #77 : App8-Bank1 : Spd=41.5km/h : Pres=30.8bar : ABS=ON : Period=500ms
[38500] #78 : App8-Bank1 : Spd=41.0km/h : Pres=31.2bar : ABS=ON : Period=500ms
[39000] #79 : App8-Bank1 : Spd=40.5km/h : Pres=31.6bar : ABS=ON : Period=500ms
[39500] #80 : App8-Bank1 : Spd=40.0km/h : Pres=32.0bar : ABS=ON : Period=500ms
[40000] #81 : App8-Bank1 : Spd=39.5km/h : Pres=32.4bar : ABS=ON : Period=500ms
[40500] #82 : App8-Bank1 : Spd=39.0km/h : Pres=32.7bar : ABS=ON : Period=500ms
[41000] #83 : App8-Bank1 : Spd=38.5km/h : Pres=33.2bar : ABS=ON : Period=500ms
[41500] #84 : App8-Bank1 : Spd=38.0km/h : Pres=33.6bar : ABS=ON : Period=500ms
[42000] #85 : App8-Bank1 : Spd=37.5km/h : Pres=34.0bar : ABS=ON : Period=500ms
[42500] #86 : App8-Bank1 : Spd=37.0km/h : Pres=34.4bar : ABS=ON : Period=500ms
[43000] #87 : App8-Bank1 : Spd=36.5km/h : Pres=34.7bar : ABS=ON : Period=500ms
[43500] #88 : App8-Bank1 : Spd=36.0km/h : Pres=35.2bar : ABS=ON : Period=500ms
[44000] #89 : App8-Bank1 : Spd=35.5km/h : Pres=35.6bar : ABS=ON : Period=500ms
[44500] #90 : App8-Bank1 : Spd=35.0km/h : Pres=36.0bar : ABS=ON : Period=500ms
[45000] #91 : App8-Bank1 : Spd=34.5km/h : Pres=36.4bar : ABS=ON : Period=500ms
[45500] #92 : App8-Bank1 : Spd=34.0km/h : Pres=36.7bar : ABS=ON : Period=500ms
[46000] #93 : App8-Bank1 : Spd=33.5km/h : Pres=37.2bar : ABS=ON : Period=500ms
[46500] #94 : App8-Bank1 : Spd=33.0km/h : Pres=37.6bar : ABS=ON : Period=500ms
[47000] #95 : App8-Bank1 : Spd=32.5km/h : Pres=38.0bar : ABS=ON : Period=500ms
[47500] #96 : App8-Bank1 : Spd=32.0km/h : Pres=38.4bar : ABS=ON : Period=500ms
[48000] #97 : App8-Bank1 : Spd=31.5km/h : Pres=38.7bar : ABS=ON : Period=500ms
[48500] #98 : App8-Bank1 : Spd=31.0km/h : Pres=39.2bar : ABS=ON : Period=500ms
Bootloader running
Current Version: 8 Other Version: 9
Bank 1 Active
SecureBoot: Starting...
SecureBoot: SHA256 FAILED
SecureBoot: BOOT BLOCKED
SecureBoot: No trusted firmware

```

Figure 19 Test case 9b(ii): Secure boot integrity verification failed due to invalid SHA-256 hash

10) Boot failed due to invalid ECDSA signature.

a) Both banks valid firmware

Initially, both bank contains valid firmware as shown in Figure 20. Firmware signature stored in active bank metadata is modified. During bootup when the signature of the active bank is checked there is mismatch. As active bank authentication failed, bank is switched and inactive bank firmware executes as shown in Figure 21.

```
COM4 - Tera Term VT
File Edit Setup Control Window Help
Bootloader running
Current Version: i1 Other Version: 10
Bank 2 Active
SecureBoot: Starting...
SecureBoot: SHA256 PASSED
SecureBoot: ECDSA PASSED
SecureBoot: Firmware TRUSTED

=== BOOT TIMING ===
Firmware size : 100788 bytes
SHA256: 6ms
ECDSA: 944ms
Total: 965ms
=====

SecureBoot: Jumping to application
Application11 Bank2 Running
[00000] #1 : App11-Bank2 : Spd=79.5km/h : Pres=0.4bar : ABS=off : Period=0ms
[00500] #2 : App11-Bank2 : Spd=79.0km/h : Pres=0.8bar : ABS=off : Period=500ms
[01000] #3 : App11-Bank2 : Spd=78.5km/h : Pres=1.2bar : ABS=off : Period=500ms
[01500] #4 : App11-Bank2 : Spd=78.0km/h : Pres=1.6bar : ABS=off : Period=500ms
[02000] #5 : App11-Bank2 : Spd=77.5km/h : Pres=2.0bar : ABS=off : Period=500ms
[02500] #6 : App11-Bank2 : Spd=77.0km/h : Pres=2.4bar : ABS=off : Period=500ms
[03000] #7 : App11-Bank2 : Spd=76.5km/h : Pres=2.7bar : ABS=off : Period=500ms
[03500] #8 : App11-Bank2 : Spd=76.0km/h : Pres=3.2bar : ABS=off : Period=500ms
[04000] #9 : App11-Bank2 : Spd=75.5km/h : Pres=3.6bar : ABS=off : Period=500ms
[04500] #10 : App11-Bank2 : Spd=75.0km/h : Pres=4.0bar : ABS=off : Period=500ms
[05000] #11 : App11-Bank2 : Spd=74.5km/h : Pres=4.4bar : ABS=off : Period=500ms
[05500] #12 : App11-Bank2 : Spd=74.0km/h : Pres=4.8bar : ABS=off : Period=500ms
[06000] #13 : App11-Bank2 : Spd=73.5km/h : Pres=5.2bar : ABS=off : Period=500ms
[06500] #14 : App11-Bank2 : Spd=73.0km/h : Pres=5.5bar : ABS=off : Period=500ms
[07000] #15 : App11-Bank2 : Spd=72.5km/h : Pres=6.0bar : ABS=off : Period=500ms
[07500] #16 : App11-Bank2 : Spd=72.0km/h : Pres=6.4bar : ABS=off : Period=500ms
[08000] #17 : App11-Bank2 : Spd=71.5km/h : Pres=6.8bar : ABS=off : Period=500ms
[08500] #18 : App11-Bank2 : Spd=71.0km/h : Pres=7.2bar : ABS=off : Period=500ms
[09000] #19 : App11-Bank2 : Spd=70.5km/h : Pres=7.5bar : ABS=off : Period=500ms
[09500] #20 : App11-Bank2 : Spd=70.0km/h : Pres=8.0bar : ABS=off : Period=500ms
[10000] #21 : App11-Bank2 : Spd=69.5km/h : Pres=8.4bar : ABS=off : Period=500ms
[10500] #22 : App11-Bank2 : Spd=69.0km/h : Pres=8.8bar : ABS=off : Period=500ms
[11000] #23 : App11-Bank2 : Spd=68.5km/h : Pres=9.1bar : ABS=off : Period=500ms
[11500] #24 : App11-Bank2 : Spd=68.0km/h : Pres=9.6bar : ABS=off : Period=500ms
[12000] #25 : App11-Bank2 : Spd=67.5km/h : Pres=10.0bar : ABS=off : Period=500ms
[12500] #26 : App11-Bank2 : Spd=67.0km/h : Pres=10.4bar : ABS=off : Period=500ms
[13000] #27 : App11-Bank2 : Spd=66.5km/h : Pres=10.8bar : ABS=off : Period=500ms
```

Figure 20 Test case 10a(i): Secure boot authentication failed due to invalid ECDSA signature

```

COM4 - Tera Term VT
File Edit Setup Control Window Help
[78500] #158 : App11-Bank2 : Spd=1.0km/h : Pres=63.2bar : ABS=ON : Period=500ms
[79000] #159 : App11-Bank2 : Spd=0.5km/h : Pres=63.6bar : ABS=ON : Period=500ms
[79500] #160 : App11-Bank2 : Spd=0.0km/h : Pres=64.0bar : ABS=ON : Period=500ms
Bootloader running
Current Version: 11 Other Version: 10
Bank 2 Active
SecureBoot: Starting...
SecureBoot: SHA256 PASSED
SecureBoot: ECDSA FAILED
SecureBoot: BOOT BLOCKED
SecureBoot: Trying other bank
OB Program Success
Launching OB Reload...
Bootloader running
Current Version: 10 Other Version: 11
Bank 1 Active
SecureBoot: Starting...
SecureBoot: SHA256 PASSED
SecureBoot: ECDSA PASSED
SecureBoot: Firmware TRUSTED

=== BOOT TIMING ===
Firmware size : 100788 bytes
SHA256: 6ms
ECDSA: 944ms
Total: 965ms
=====
SecureBoot: Jumping to application
Application10 Bank1 Running
[00000] #1 : App10-Bank1 : Spd=79.5km/h : Pres=0.4bar : ABS=off : Period=0ms
[00500] #2 : App10-Bank1 : Spd=79.0km/h : Pres=0.8bar : ABS=off : Period=500ms
[01000] #3 : App10-Bank1 : Spd=78.5km/h : Pres=1.2bar : ABS=off : Period=500ms
[01500] #4 : App10-Bank1 : Spd=78.0km/h : Pres=1.6bar : ABS=off : Period=500ms
[02000] #5 : App10-Bank1 : Spd=77.5km/h : Pres=2.0bar : ABS=off : Period=500ms
[02500] #6 : App10-Bank1 : Spd=77.0km/h : Pres=2.4bar : ABS=off : Period=500ms
[03000] #7 : App10-Bank1 : Spd=76.5km/h : Pres=2.7bar : ABS=off : Period=500ms
[03500] #8 : App10-Bank1 : Spd=76.0km/h : Pres=3.2bar : ABS=off : Period=500ms
[04000] #9 : App10-Bank1 : Spd=75.5km/h : Pres=3.6bar : ABS=off : Period=500ms
[04500] #10 : App10-Bank1 : Spd=75.0km/h : Pres=4.0bar : ABS=off : Period=500ms
[05000] #11 : App10-Bank1 : Spd=74.5km/h : Pres=4.4bar : ABS=off : Period=500ms
[05500] #12 : App10-Bank1 : Spd=74.0km/h : Pres=4.8bar : ABS=off : Period=500ms
[06000] #13 : App10-Bank1 : Spd=73.5km/h : Pres=5.2bar : ABS=off : Period=500ms
[06500] #14 : App10-Bank1 : Spd=73.0km/h : Pres=5.5bar : ABS=off : Period=500ms

```

Figure 21 Test case 10a(ii): Secure boot authentication failed due to invalid ECDSA signature

b) Single bank valid firmware

Initially only active bank contains valid firmware as shown in Figure 22. Firmware signature stored in active bank metadata is modified. During bootup when the signature of the active bank is checked there is mismatch. Active bank authentication failed, and inactive bank does not contain valid firmware. As there are no valid firmware, bootloader remains in a waiting state until a valid firmware image is manually flashed as shown in Figure 23.

```
COM4 - Tera Term VT
File Edit Setup Control Window Help
Bootloader running
Current Version: 10 Other Version: 11
Bank 1 Active
SecureBoot: Starting...
SecureBoot: SHA256 PASSED
SecureBoot: ECDSA PASSED
SecureBoot: Firmware TRUSTED
=== BOOT TIMING ===
Firmware size : 100788 bytes
SHA256: 6ms
ECDSA: 944ms
Total: 965ms
=====
SecureBoot: Jumping to application
Application10 Bank1 Running
[00000] #1 App10-Bank1 Spd=79.5km/h Pres=0.4bar ABS=off Period=0ms
[00500] #2 App10-Bank1 Spd=79.0km/h Pres=0.8bar ABS=off Period=500ms
[01000] #3 App10-Bank1 Spd=78.5km/h Pres=1.2bar ABS=off Period=500ms
[01500] #4 App10-Bank1 Spd=78.0km/h Pres=1.6bar ABS=off Period=500ms
[02000] #5 App10-Bank1 Spd=77.5km/h Pres=2.0bar ABS=off Period=500ms
[02500] #6 App10-Bank1 Spd=77.0km/h Pres=2.4bar ABS=off Period=500ms
[03000] #7 App10-Bank1 Spd=76.5km/h Pres=2.7bar ABS=off Period=500ms
[03500] #8 App10-Bank1 Spd=76.0km/h Pres=3.2bar ABS=off Period=500ms
[04000] #9 App10-Bank1 Spd=75.5km/h Pres=3.6bar ABS=off Period=500ms
[04500] #10 App10-Bank1 Spd=75.0km/h Pres=4.0bar ABS=off Period=500ms
[05000] #11 App10-Bank1 Spd=74.5km/h Pres=4.4bar ABS=off Period=500ms
[05500] #12 App10-Bank1 Spd=74.0km/h Pres=4.8bar ABS=off Period=500ms
[06000] #13 App10-Bank1 Spd=73.5km/h Pres=5.2bar ABS=off Period=500ms
[06500] #14 App10-Bank1 Spd=73.0km/h Pres=5.5bar ABS=off Period=500ms
[07000] #15 App10-Bank1 Spd=72.5km/h Pres=6.0bar ABS=off Period=500ms
[07500] #16 App10-Bank1 Spd=72.0km/h Pres=6.4bar ABS=off Period=500ms
[08000] #17 App10-Bank1 Spd=71.5km/h Pres=6.8bar ABS=off Period=500ms
[08500] #18 App10-Bank1 Spd=71.0km/h Pres=7.2bar ABS=off Period=500ms
[09000] #19 App10-Bank1 Spd=70.5km/h Pres=7.5bar ABS=off Period=500ms
[09500] #20 App10-Bank1 Spd=70.0km/h Pres=8.0bar ABS=off Period=500ms
[10000] #21 App10-Bank1 Spd=69.5km/h Pres=8.4bar ABS=off Period=500ms
[10500] #22 App10-Bank1 Spd=69.0km/h Pres=8.8bar ABS=off Period=500ms
[11000] #23 App10-Bank1 Spd=68.5km/h Pres=9.1bar ABS=off Period=500ms
[11500] #24 App10-Bank1 Spd=68.0km/h Pres=9.6bar ABS=off Period=500ms
[12000] #25 App10-Bank1 Spd=67.5km/h Pres=10.0bar ABS=off Period=500ms
[12500] #26 App10-Bank1 Spd=67.0km/h Pres=10.4bar ABS=off Period=500ms
[13000] #27 App10-Bank1 Spd=66.5km/h Pres=10.8bar ABS=off Period=500ms
```

Figure 22 Test case 10b(i): Secure boot authentication failed due to invalid ECDSA signature

```
COM4 - Tera Term VT
File Edit Setup Control Window Help
[77000] #155 App10-Bank1 Spd=2.5km/h Pres=62.0bar ABS=ON Period=500ms
[77500] #156 App10-Bank1 Spd=2.0km/h Pres=62.4bar ABS=ON Period=500ms
[78000] #157 App10-Bank1 Spd=1.5km/h Pres=62.7bar ABS=ON Period=500ms
[78500] #158 App10-Bank1 Spd=1.0km/h Pres=63.2bar ABS=ON Period=500ms
[79000] #159 App10-Bank1 Spd=0.5km/h Pres=63.6bar ABS=ON Period=500ms
[79500] #160 App10-Bank1 Spd=0.0km/h Pres=64.0bar ABS=ON Period=500ms
[80000] #161 App10-Bank1 Spd=80.0km/h Pres=0.0bar ABS=off Period=500ms
[80500] #162 App10-Bank1 Spd=79.5km/h Pres=0.4bar ABS=off Period=500ms
[81000] #163 App10-Bank1 Spd=79.0km/h Pres=0.8bar ABS=off Period=500ms
[81500] #164 App10-Bank1 Spd=78.5km/h Pres=1.2bar ABS=off Period=500ms
[82000] #165 App10-Bank1 Spd=78.0km/h Pres=1.6bar ABS=off Period=500ms
[82500] #166 App10-Bank1 Spd=77.5km/h Pres=2.0bar ABS=off Period=500ms
[83000] #167 App10-Bank1 Spd=77.0km/h Pres=2.4bar ABS=off Period=500ms
[83500] #168 App10-Bank1 Spd=76.5km/h Pres=2.7bar ABS=off Period=500ms
[84000] #169 App10-Bank1 Spd=76.0km/h Pres=3.2bar ABS=off Period=500ms
[84500] #170 App10-Bank1 Spd=75.5km/h Pres=3.6bar ABS=off Period=500ms
[85000] #171 App10-Bank1 Spd=75.0km/h Pres=4.0bar ABS=off Period=500ms
[85500] #172 App10-Bank1 Spd=74.5km/h Pres=4.4bar ABS=off Period=500ms
[86000] #173 App10-Bank1 Spd=74.0km/h Pres=4.8bar ABS=off Period=500ms
[86500] #174 App10-Bank1 Spd=73.5km/h Pres=5.2bar ABS=off Period=500ms
[87000] #175 App10-Bank1 Spd=73.0km/h Pres=5.5bar ABS=off Period=500ms
[87500] #176 App10-Bank1 Spd=72.5km/h Pres=6.0bar ABS=off Period=500ms
[88000] #177 App10-Bank1 Spd=72.0km/h Pres=6.4bar ABS=off Period=500ms
[88500] #178 App10-Bank1 Spd=71.5km/h Pres=6.8bar ABS=off Period=500ms
[89000] #179 App10-Bank1 Spd=71.0km/h Pres=7.2bar ABS=off Period=500ms
[89500] #180 App10-Bank1 Spd=70.5km/h Pres=7.5bar ABS=off Period=500ms
[90000] #181 App10-Bank1 Spd=70.0km/h Pres=8.0bar ABS=off Period=500ms
[90500] #182 App10-Bank1 Spd=69.5km/h Pres=8.4bar ABS=off Period=500ms
[91000] #183 App10-Bank1 Spd=69.0km/h Pres=8.8bar ABS=off Period=500ms
[91500] #184 App10-Bank1 Spd=68.5km/h Pres=9.1bar ABS=off Period=500ms
[92000] #185 App10-Bank1 Spd=68.0km/h Pres=9.6bar ABS=off Period=500ms
[92500] #186 App10-Bank1 Spd=67.5km/h Pres=10.0bar ABS=off Period=500ms
[93000] #187 App10-Bank1 Spd=67.0km/h Pres=10.4bar ABS=off Period=500ms
[93500] #188 App10-Bank1 Spd=66.5km/h Pres=10.8bar ABS=off Period=500ms
[94000] #189 App10-Bank1 Spd=66.0km/h Pres=11.1bar ABS=off Period=500ms
Bootloader running
Current Version: 10 Other Version: 11
Bank 1 Active
SecureBoot: Starting...
SecureBoot: SHA256 PASSED
SecureBoot: ECDSA FAILED
SecureBoot: BOOT BLOCKED
SecureBoot: No trusted firmware
```

Figure 23 Test case 10b(ii): Secure boot authentication failed due to invalid ECDSA signature

4.5.Summary

The results confirm that the implemented system meets all stated objectives. Hardware acceleration delivers a 16.3x improvement in SHA-256 boot-time verification and a 6.4x improvement in AES-256 OTA decryption throughput, demonstrating a quantifiable performance benefit for hardware cryptographic peripherals in resource-constrained automotive ECUs. ECDSA verification remains in software due to the absence of elliptic curve hardware support on the STM32H753ZI, and its execution time of approximately 940 ms represents the primary latency contribution outside of flash operations. All functional test scenarios including security rejection, watchdog rollback, and incomplete transfer recovery produced the expected outcomes, validating the reliability and security properties of the implemented system.

5. Conclusion and Future Scope of Work

This dissertation presents the design and implementation of a secure and reliable Over-the-Air firmware update system for embedded controllers, targeting the STM32H753ZI microcontroller with an ESP32 Wi-Fi bridge. The implemented system integrates a custom bare-metal bootloader with dual-bank flash management, a hardware-accelerated cryptographic security pipeline (AES-256-CTR via the CRYPT peripheral and SHA-256 via the HASH peripheral), software-based ECDSA P-256 authentication using mbedTLS, a custom binary UART protocol with stop-and-wait ARQ, and a watchdog-driven rollback mechanism, all validated on real hardware against a FreeRTOS application simulating an automotive brake ECU workload.

Performance measurements confirm that for 128KB firmware image, hardware acceleration reduces SHA-256 boot verification time by 16.3x (from 147 ms to 9 ms) and AES-256 OTA decryption time by 6.4x (from 218.6 ms to 34.3 ms per 128 KB payload). Functional test cases covering normal OTA operation, security rejection under corrupted hash, signature, and nonce, watchdog rollback from a runtime-failing firmware, incomplete transfer recovery, and bootloader behaviour across single-bank and dual-bank valid/invalid combinations, confirming the correctness and robustness of the implementation.

Several extensions are identified for extending this work beyond scope of the current prototype implementation.

- Replacing UART with CAN or CAN-FD as the inter-ECU transport, which is the dominant physical layer in production automotive networks and would bring the prototype closer to real deployment conditions.

- Automotive Ethernet integration for higher-bandwidth firmware delivery, enabling OTA for larger firmware images without the throughput constraints of the UART link.
- Hardware security module integration for key storage and key management.
- Multi-device fleet management by extending the local OTA server to support structure management and automated rollout of firmware.

6. Abbreviations

Table 6 Abbreviations

Abbreviations	Expanded form
OTA	Over-the-Air
ECU	Electronic Control Unit
ABS	Anti-lock Braking System
AES	Advanced Encryption Standard
ECDSA	Elliptic Curve Digital Signature Algorithm
SHA	Secure Hash Algorithm
Wi-Fi	Wireless Fidelity
RSA	Rivest-Shamir-Adleman
ARQ	Automatic Repeat Request
UART	Universal Asynchronous Receiver Transmitter
CRC-CCITT	Cyclic Redundancy Check - Consultative Committee for International Telephony and Telegraphy

7. References

- [1] T. Mirfakhraie, G. Vitor and K. Grogan, "Applicable Protocol for Updating Firmware of Automotive HVAC Electronic Control Units (ECUs) Over the Air," in *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, Halifax, NS, Canada, 2018.
- [2] O. Kachman and M. Balaz, "Efficient Patch Module for Single-bank or Dual-bank Firmware Updates for Embedded Devices," in *2020 23rd International Symposium on Design and Diagnostics of Electronic Circuits & Systems (DDECS)*, Novi Sad, Serbia, 2020.
- [3] S. K, J. S, J. A. Kovilpillai J, S. Santhosh S and S. R, "Efficient Implementation of Firmware Over-the-Air Update using Raspberry Pi 4 and STM32F103C8T6," in *2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECCT)*, Bangalore, India, 2024.
- [4] M. Kubaščík, I. A. Tupý, J. Šumský and T. Bača, "OTA firmware updates on ESP32 based microcontrollers," in *2024 IEEE 17th International Scientific Conference on Informatics (Informatics)*, Poprad, Slovakia, 2024.
- [5] O. Sangrulkar, S. Motade, K. Gunale and A. Deshpande, "Implementation of Secure and Scalable Over-the-Air Firmware Updates for STM32F407VG with Quectel M66 and Cloud Integration," in *2025 12th International Conference on Emerging Trends in Engineering & Technology - Signal and Information Processing (ICETET - SIP)*, Nagpur, India, 2025.

- [6] J. Wang, R. Chen, H. Wen, Y. Chen, Q. Hou and W. Duan, "A Secure and Efficient Vehicle OTA Software Update System: Design and Validation," in *2024 6th International Conference on Electronic Engineering and Informatics (EEI)*, Chongqing, China, 2024.

Checklist of Items for the Final Dissertation / Project / Project Work Report

This checklist is to be attached as the last page of the final report.

This checklist is to be duly completed, verified and signed by the student.

1.	Is the final report neatly formatted with all the elements required for a technical Report?	Yes
2.	Is the Cover page in proper format as given in Annexure A?	Yes
3.	Is the Title page (Inner cover page) in proper format?	Yes
4.	(a) Is the Certificate from the Supervisor in proper format? (b) Has it been signed by the Supervisor?	Yes Yes
5.	Is the Abstract included in the report properly written within one page? Have the technical keywords been specified properly?	Yes Yes
6.	Is the title of your report appropriate? The title should be adequately descriptive, precise and must reflect scope of the actual work done. Uncommon abbreviations / Acronyms should not be used in the title	Yes
7.	Have you included the List of abbreviations / Acronyms?	Yes
8.	Does the Report contain a summary of the literature survey?	Yes
9.	Does the Table of Contents include page numbers? (i). Are the Pages numbered properly? (Ch. 1 should start on Page # 1) (ii). Are the Figures numbered properly? (Figure Numbers and Figure Titles should be at the bottom of the figures) (iii). Are the Tables numbered properly? (Table Numbers and Table Titles should be at the top of the tables) (iv). Are the Captions for the Figures and Tables proper? (v). Are the Appendices numbered properly? Are their titles appropriate	Yes Yes Yes Yes Yes Yes
10.	Is the conclusion of the Report based on discussion of the work?	Yes
11.	Are References or Bibliography given at the end of the Report? Have the References been cited properly inside the text of the Report? Are all the references cited in the body of the report	Yes Yes Yes
12.	Is the report format and content according to the guidelines? The report should not be a mere printout of a PowerPoint Presentation, or a user manual. Source code of software need not be included in the report.	Yes

Declaration by Student:

I certify that I have properly verified all the items in this checklist and ensure that the report is in proper format as specified in the course handout.

A handwritten signature in blue ink that reads "Varad". The signature is written in a cursive style with a long horizontal stroke extending to the right.

Place: Bengaluru

Signature of the Student

Date: 11th May 2026

Name: Varad Vinod Prabhu

ID No.: 2024HT01040